ChartNav — Consolidated Build

Backend phases 1–16: spine → state machine → auth → RBAC → CI → production seam → Postgres → frontend → E2E → JWT + ops → staging → governance → control plane → invitations → enterprise quality & compliance → platform mode & interoperability
Repo: `thekidd2227/chartnav-platform`

Contents

1. Executive summary
2. 01 — Current state
3. 02 — Workflow state machine
4. 03 — API endpoints
5. 04 — Data model
6. 05 — Build log
7. 06 — Known gaps + verification matrix
8. 07 — Auth, org scoping, RBAC
9. 08 — Test strategy
10. 09 — CI & deploy hardening
11. 10 — Doc artifact pipeline
12. 11 — Production-auth seam
13. 12 — Runtime config
14. 13 — Deploy target
15. 14 — Postgres parity
16. 15 — Frontend integration
17. 16 — Frontend test strategy
18. 17 — E2E & release
19. 18 — Operational hardening
20. 19 — Staging deployment
21. 20 — Observability
22. 21 — Staging runbook
23. 22 — Admin governance
24. 23 — Operator control plane
25. 24 — Invitations & governance
26. 25 — Enterprise quality & compliance
27. 26 — Platform mode & interoperability
28. 27 — Adoption & implementation model
29. Diagram — System architecture
30. Diagram — Encounter status machine
31. Diagram — ER
32. Diagram — API / data flow

Executive summary

This document consolidates **all** ChartNav backend phases delivered to

date.

Phase 1 — Workflow spine. Alembic migration `a1b2c3d4e5f6` introduced `encounters` and `workflow_events` tables with FKs and indexes. Six encounter endpoints. Idempotent seed.

Phase 2 — State machine + filtering. `POST /encounters/{id}/status` enforces explicit forward + rework edges. `GET /encounters` accepts `organization_id`, `location_id`, `status`, `provider_name` filters.

Phase 3 — Dev auth + org scoping. `apps/api/app/auth.py` resolves the caller from `X-User-Email`. Every encounter route became org-scoped: cross-org reads return 404, cross-org body/query assertions 403.

Phase 4 — RBAC + full scoping + tests. `apps/api/app/authz.py` introduces `admin` / `clinician` / `reviewer` roles and per-edge transition rules. `/organizations`, `/locations`, `/users` are now authenticated and scoped. `{error_code, reason}` envelope is standardized. 25-test pytest suite covers auth, scoping, RBAC, state-machine invariants.

Phase 5 — CI + runtime hardening. GitHub Actions workflow runs install + Alembic upgrade on an isolated CI DB + idempotent seed + pytest + live smoke. A separate `docs` job regenerates the final HTML/PDF. Root `Makefile`, `apps/api/scripts/smoke.sh`, and `scripts/build_docs.py` codify the local verification + doc-build paths.

Phase 6 — Production seam + deploy target + Postgres parity. DB layer moved to SQLAlchemy Core. `apps/api/app/config.py` centralizes env. `auth.py` gained a bearer stub that returns 501 honestly. Dockerfile hardened, `docker-compose.prod.yml` + `scripts/pg_verify.sh` + `backend-postgres` CI job land.

Phase 7 — Frontend workflow UI. Typed API client + identity seam + full two-pane workflow console (list + filters + detail + timeline + role-aware actions). Error banners surface the backend envelope verbatim.

Phase 8 — Create UI + frontend tests + frontend CI. Admin/clinician can create encounters through a modal; Vitest + Testing Library harness (12 tests) locks the UI down; dedicated `frontend` CI job runs typecheck + tests + build on every push/PR.

Phase 9 — Playwright E2E + release pipeline. Playwright boots backend + frontend together, 8 Chromium scenarios, new `e2e` CI job. `release.yml` workflow on `v*.*.*` tags: GHCR push + release bundle + GitHub Release with tarballs and MANIFEST.

Phase 10 — Real JWT bearer auth + operational hardening. Real PyJWT validation against a JWKS URL (signature + iss + aud + exp + claim mapping). Request correlation (`X-Request-ID`), structured JSON logs, `security_audit_events` table (`b2c3d4e5f6a7`), CORS driven by config, per-process rate limiter.

Phase 11 — Staging deployment + observability. New `/ready` + `/metrics` surfaces. Pinned-image staging compose + runbook scripts + release-bundled staging tarball. `deploy-config` CI job validates compose + shellcheck.

Phase 12 — Admin governance + event discipline + pagination. DB CHECK on `users.role`, `is_active` soft-delete flags, admin CRUD for users + locations, `EVENT_SCHEMAS` allowlist, encounter pagination via `limit` / `offset` + `X-Total-Count`. `AdminPanel` modal with Users + Locations tabs.

Phase 13 — Operator control plane. `GET/PATCH /organization` + `GET /security-audit-events` + admin panel Organization & Audit tabs. `invited_at` stamping on admin create.

Phase 14 — Invitations, settings schema, audit export, event hardening, bulk users. Migration `e5f6a7b8c9d0` adds `invitation_token_hash` (sha256-only), `invitation_expires_at`, `invitation_accepted_at`. New endpoints: `POST /users/{id}/invite`, `POST /invites/accept`, `POST /users/bulk`, `GET /security-audit-events/export`. Organization settings becomes a typed `OrganizationSettings` pydantic model (extra=forbid) with dedicated fields + an `extensions` forward-compat bucket. Event payloads enforce per-type value discipline (status enums, non-empty strings, non-negative ints). Admin UI adds per-user **Invite** with a one-shot raw-token banner, a **Bulk import...** dialog, an **Export CSV** button on the audit tab, typed inputs on the Organization tab, and a minimal `/invite?invite=<token>` accept screen. Suites grow to **110 pytest / 25 Vitest / 12 Playwright**.

Phase 15 — Enterprise quality + compliance. Five real enterprise-credibility guarantees land together. (1) **Accessibility baseline**: `@axe-core/playwright` over Chromium, 5 scans covering app shell + encounter list/detail + admin users/audit + invite accept; `serious` / `critical` findings are blocking; established fixes land `aria-label` on the event-type and inline role `<select>` s. (2) **Visual regression baseline**: 4 Playwright screenshots (encounter list, admin users, admin audit, invite accept) at 1280×820 with animations disabled and `maxDiffPixelRatio: 0.02`; macOS baselines committed for local dev; CI intentionally skips because Chromium renders differently on Linux (documented gap). (3) **Admin list scaling**: `GET /users` and `GET /locations` gain `limit`, `offset`, `q`, and (users) `role` query params with `X-Total-Count` / `X-Limit` / `X-Offset` headers; UI adds search + Prev/Next pager (25/page) on both tabs. (4) **Feature-flag consumption**: `organization.settings.feature_flags.audit_export` and `.bulk_import` now actually hide the **Export CSV** / **Bulk import...** buttons; defaults stay `true`; documented as a UX toggle, not a security control. (5) **Release compliance signals**: `scripts/sbom.py` captures project + git sha/tag/dirty + image tag + `pip list` + `npm list`; `scripts/release_build.sh` also emits `chartnav-api-<v>.digest.txt` via `docker image inspect`; both are attached to GitHub Releases and included in `MANIFEST.txt`. (6) **Audit retention scaffolding**: `apps/api/app/retention.py::prune_audit_events` + `scripts/audit_retention.py` CLI + `CHARTNAV_AUDIT_RETENTION_DAYS` env (default `0` = never); the app never silently prunes. Suites grow to **118 pytest / 28 Vitest / 21 Playwright** (12 workflow + 5 a11y + 4 visual-local).

Phase 16 — Platform mode & interoperability (this phase). ChartNav is now explicitly two products in one — a standalone lightweight EMR/EHR **and** an integration-first overlay on top of an existing EHR/EMR. The shape is architectural, not a feature flag bolted on. Four real changes landed together: (1) **Operating modes**: `CHARTNAV_PLATFORM_MODE` ∈ { `standalone`, `integrated_readthrough`, `integrated_writethrough` }, validated at import time; `standalone` pins the adapter to `native`, `integrated_*` defaults to `stub`. (2) **Adapter boundary** — `app/integrations/base.py::ClinicalSystemAdapter` protocol (`fetch_patient`, `search_patients`, `fetch_encounter`, `update_encounter_status`, `write_note`, `sync_reference_data`, `info`), `AdapterError` / `AdapterNotSupported`, `SourceOfTruth` enum. Ships two honest implementations: `NativeChartNavAdapter` (persists to the ChartNav DB via the same SA Core surface the HTTP routes use, refuses patient ops until a native `patients` table lands), and `StubClinicalSystemAdapter` (canned reads, write-through records writes to an in-process list, read-through raises `AdapterNotSupported`). Vendor adapters plug into a mutable `_VENDOR_ADAPTERS` registry via `register_vendor_adapter(key, factory)` — no decorator magic, grep-able. (3) **HTTP surface** — `GET /platform` (any authenticated caller) returns mode + adapter key + display name + supports-* flags + source-of-truth map. No secret leakage (asserted). (4) **Frontend awareness** — `api.ts` gains `getPlatform` + types; `AdminPanel` fetches `/platform` on mount and renders a platform banner above the tabs on every admin view. CI fallout from phase 15 was repaired in the same commit window: migration `c3d4e5f6a7b8` switched from `sa.text("1")` (Postgres-incompatible BOOLEAN default) to `sa.text("true")`, and vitest was downgraded to `^3.2.4` + lockfile regenerated to fix `npm ci` on Linux/Node 20 (Vitest 4 had transitively pulled rolldown + Vite 8 + esbuild 0.28, leaving the lockfile incomplete for non-mac platforms). Suites grow to **131 pytest** / **30 Vitest** / 21 Playwright. `GET /platform` is the only new endpoint.

Preserved untouched: `/health`, `/`, SQLite dev workflow, state machine + filtering surface, workflow_events model, existing endpoint contracts.

ChartNav — Current State

As of: 2026-04-18 (phase: platform mode & interoperability)

Repo layout (relevant)

```
chartnav-platform/
├── .github/workflows/{ci.yml,release.yml}
├── Makefile
├── scripts/                # build_docs · verify · pg_verify · release_build · staging_*
│                           # audit_retention · sbom
├── apps/
│   ├── api/
│   │   ├── app/
│   │   │   ├── main.py · config.py · db.py · auth.py · authz.py
│   │   │   ├── audit.py · retention.py · logging_config.py · middleware.py · metrics.py
│   │   │   ├── integrations/    # adapter boundary (phase 16)
│   │   │   │   ├── __init__.py  # resolve_adapter + vendor registry
│   │   │   │   ├── base.py      # ClinicalSystemAdapter protocol + errors
│   │   │   │   ├── native.py    # NativeChartNavAdapter
│   │   │   │   └── stub.py      # StubClinicalSystemAdapter
│   │   │   └── api/routes.py    # + GET /platform (phase 16)
│   │   ├── alembic/versions/    # 6 migrations through e5f6a7b8c9d0
│   │   ├── tests/               # 131 pytest (+13 test_platform_mode)
│   │   └── Dockerfile · entrypoint.sh · .env.example
│   └── web/
│       ├── src/
│       │   ├── App.tsx · AdminPanel.tsx · InviteAccept.tsx · api.ts
│       │   ├── identity.ts · styles.css · main.tsx
│       │   └── test/            # 30 Vitest (+2 platform-mode UI)
│       └── tests/e2e/
│           ├── workflow.spec.ts (12)
│           ├── a11y.spec.ts (5)   # axe-core — hard gate in CI
│           └── visual.spec.ts (4) # screenshot regression — local only
├── infra/docker/{dev,prod,staging}.yml
└── docs/build/ 01 … 27
```

Runtime baseline

- Backend: FastAPI + SQLAlchemy Core + PyJWT.
- Frontend: Vite 5 + React 18 + TypeScript + Vitest + Playwright.
- Auth: `header` (dev) or `bearer` (prod JWT via JWKS).
- RBAC: `admin` / `clinician` / `reviewer` (CHECK-constrained).
- Error envelope: `{"detail": {"error_code": "...", "reason": "..."}}` .
- **a11y**: axe-core baseline covering app shell, encounter list/detail, admin panel, invite page. `serious` / `critical` violations are blocking.
- **Visual regression**: 4 macOS-baseline screenshots; `maxDiffPixelRatio: 0.02` . Local gate; CI skips (OS-specific pixels).
- **Admin list scaling**: `GET /users` / `/locations` now take `limit` , `offset` , `q` , role (+users). `X-Total-Count` headers. UI has search + Prev/Next pagers on Users + Locations tabs.
- **Feature flags**: `organization.settings.feature_flags.audit_export` and `.bulk_import` actually gate the corresponding admin buttons.
- **Audit retention**: `scripts/audit_retention.py` prunes old rows per `CHARTNAV_AUDIT_RETENTION_DAYS` ; the app never silently prunes.
- **Release compliance**: SBOM (`chartnav-sbom-<v>.json`) + image digest (`chartnav-api-<v>.digest.txt`) in every release bundle, attached to the GitHub Release.
- **Platform mode** (phase 16): `CHARTNAV_PLATFORM_MODE` ∈ { `standalone` , `integrated_readthrough` , `integrated_writethrough` }. Adapter boundary (`ClinicalSystemAdapter`) separates ChartNav core from any external EHR/EMR. Ships `NativeChartNavAdapter` (standalone) + `StubClinicalSystemAdapter` (integrated placeholder); vendor adapters plug in via `register_vendor_adapter` . Config fails loudly on misconfig. `GET /platform` surfaces mode + adapter + source-of-truth to the UI.
- Alembic head: `e5f6a7b8c9d0` .

Testing layers

Layer	Tool	Count	Notes
pytest	pytest	131	+13 platform-mode + adapter resolution
shell smoke	smoke.sh	9	unchanged
Vitest	vitest	30	+2 platform-mode banner (standalone + integrated variants)
Playwright workflow	@playwright	12	unchanged contract
Playwright a11y	@axe-core/playwright	5	NEW
Playwright visual	Playwright snapshots	4	NEW (local only)
staging	staging_verify.sh	9	unchanged

Verified working endpoints

No new endpoints this phase. `GET /users` and `GET /locations` gained

`q` / `role` / `limit` / `offset` query parameters + pagination headers;

older callers that pass no params still see the first 100 rows.

Automation

- `make verify` → 131 pytest + 9 smoke
- `make web-verify` → 30 vitest + typecheck + build
- `make e2e` → 12 workflow + 5 a11y + 4 visual (local)
- `make e2e-a11y` / `make e2e-visual` / `make e2e-visual-update`
- `make audit-prune ARGS="--days 90 --dry-run"`
- `make sbom` → `dist/release/_sbom.json`
- `make pg-verify` / `make staging-*` / `make release-build VERSION=v0.1.0`
- `make dev` — backend + frontend together
- CI: backend-sqlite + frontend + deploy-config in parallel; e2e (workflow + a11y) gates on backend + frontend; backend-postgres + docker-build + docs chain on backend-sqlite.

Encounter Workflow State Machine

Status values

Status	Meaning
`scheduled`	Visit is booked, patient has not arrived.
`in_progress`	Provider is seeing the patient / actively charting.
`draft_ready`	Note draft has been generated and is awaiting review.
`review_needed`	Reviewer flagged the note — needs revision or sign-off.
`completed`	Note signed off; encounter closed. **Terminal.**

Allowed transitions

Forward flow:

```
scheduled → in_progress → draft_ready → review_needed → completed
```

Explicit rework transitions (documented + enforced):

From	To	Purpose
`draft_ready`	`in_progress`	Draft rejected, return to charting.
`review_needed`	`draft_ready`	Reviewer kicks note back for rewrite.

Rejected transitions (examples that return HTTP 400)

- `scheduled → completed`
- `scheduled → review_needed`
- `scheduled → draft_ready`
- `in_progress → scheduled`
- `in_progress → completed` (must pass through draft_ready + review_needed)
- `completed → anything` (terminal)
- Any unknown status string (`invalid_status`)

Error body format:

```
{"detail":"invalid_transition: in_progress -> completed is not permitted; allowed next states from in_progress: ['draft_ready'
```

Idempotency

Posting the **same** status the encounter is already in is a no-op: the row

is returned unchanged and **no** `status_changed` event is appended.

Side effects on successful transition

Transition target	Timestamp action
`in_progress`	set `started_at = now()` if currently NULL

Transition target	Timestamp action
`completed`	set `completed_at = now()` ; also set `started_at = now()` if it was NULL
other	no timestamp mutation

Every successful transition appends a `workflow_events` row:

```
{
  "event_type": "status_changed",
  "event_data": {"old_status": "...", "new_status": "..."}
}
```

Diagram

See `docs/diagrams/encounter-status-machine.md` (Mermaid).

Event payload hardening (phase 14)

On top of the phase-12 "required keys present" check,

`_validate_event` now enforces per-type value discipline:

- `status_changed` : `old_status` and `new_status` must both be in `ALLOWED_STATUSES` .
- `encounter_created` : `status` must be in `ALLOWED_STATUSES` .
- `manual_note` : `note` must be a non-empty string ≤ 4000 chars.
- `note_draft_requested` : `requested_by` non-empty string; optional `template` non-empty string.
- `note_draft_completed` : `template` non-empty string; optional `length_words` non-negative int.
- `note_reviewed` : `reviewer` non-empty string ≤ 255.

Violations return 400 `invalid_event_data` with a specific reason.

Source of truth

`apps/api/app/api/routes.py` :

- `ALLOWED_STATUSES` — the 5 permitted status strings.
- `ALLOWED_TRANSITIONS` — `dict[str, set[str]]` , one entry per source state.
- `update_encounter_status()` — the single enforcement point.

API Endpoints

Base URL (local dev): `http://127.0.0.1:8000` . All error bodies use

`{"detail": {"error_code": "...", "reason": "..."}}` .

System (open)

Method	Path
GET	`/health`
GET	`/`

Identity

Method	Path	Auth	Roles
GET	`/me`	🔒	any

Org metadata (🔒, org-scoped)

Method	Path	Behavior
GET	`/organizations`	Returns only caller's org row.
GET	`/locations`	`WHERE organization_id = caller.org` . Supports `limit` (1..500, default 100), `offset` (≥0), `q` (substring search on `name`), plus `include_inactive` (admin). Emits `X-Total-Count` , `X-Limit` , `X-Offset` headers.
GET	`/users`	`WHERE organization_id = caller.org` . Supports `limit` , `offset` , `q` (substring on `email` or `full_name`), `role` (valid roles only — invalid → 400 `invalid_role`), `include_inactive` (admin). Emits `X-Total-Count` , `X-Limit` , `X-Offset` headers.

Encounters (🔒, org-scoped + RBAC)

GET `/encounters`

Any authenticated role. Always `WHERE organization_id = caller.org` .

Optional query params (AND-ed, parameterized):

Param	Rule
`organization_id`	Must equal `caller.org` — else 403.
`location_id`	Narrows within caller org.
`status`	Must be in `ALLOWED_STATUSES` — else 400.
`provider_name`	Exact match.

GET `/encounters/{id}` — any role

404 if missing or cross-org.

GET `/encounters/{id}/events` — any role

404 if parent missing or cross-org.

POST `/encounters` — admin, clinician

Body:

```
{
  "organization_id": 1,
  "location_id": 1,
  "patient_identifier": "PT-1003",
  "patient_name": "Alex Chen",
  "provider_name": "Dr. Ortiz",
  "scheduled_at": "2026-04-18T14:30:00Z",
  "status": "scheduled"
}
```

POST `/encounters/{id}/events` — admin, clinician

Reviewer → 403. 404 if cross-org.

POST `/encounters/{id}/status`

Per-edge RBAC:

From	To	Roles allowed
scheduled	in_progress	admin, clinician
in_progress	draft_ready	admin, clinician
draft_ready	in_progress	admin, clinician
draft_ready	review_needed	admin, reviewer
review_needed	draft_ready	admin, reviewer
review_needed	completed	admin, reviewer

Same-state POST = no-op (200). Role violation → 403 `role_cannot_transition`. Invalid transition → 400 `invalid_transition`. Cross-org → 404.

Admin endpoints (🔒 `admin` role only, org-scoped)

Method	Path	Body
POST	`/users`	`{email, full_name?, role}`
PATCH	`/users/{id}`	`{email?, full_name?, role?, is_active?}`
DELETE	`/users/{id}`	soft-delete (sets `is_active=0`)
POST	`/locations`	`{name}`
PATCH	`/locations/{id}`	`{name?, is_active?}`
DELETE	`/locations/{id}`	soft-delete

Error codes introduced:

- `role_admin_required` (403) — non-admin tried an admin action.
- `invalid_role` (400) — role not in `{admin, clinician, reviewer}`.
- `user_email_taken` (409) — email uniqueness conflict.
- `cannot_demote_self` (400), `cannot_deactivate_self` (400) — admin self-protection.
- `user_not_found` / `location_not_found` (404) — includes cross-org lookups (no existence leak).

See `22-admin-governance.md` for the governance model.

Organization settings (🔒 org-scoped)

Method	Path	Auth	Body
GET	`/organization`	any authed role	—
PATCH	`/organization`	admin only	`{ name?, settings? }`

Rules:

- `slug` is **immutable** — not accepted by PATCH.
- `settings` is a JSON object; non-object → 422; serialized > 16 KB → 400 `settings_too_large`.
- PATCH always scopes to caller's org — no cross-org parameter exists.

Invitations (phase 14)

Method	Path	Auth
POST	`/users/{id}/invite`	admin
POST	`/invites/accept`	unauth

Admin response returns raw `invitation_token` **once**; only the sha256 hash is stored. 7-day expiry. Accept body `{ token }`. Errors: `user_not_found` (404 cross-org), `user_inactive` (400), `user_already_accepted` (400), `invalid_invite` (400), `invite_expired` (400).

Bulk user import

Method	Path	Auth
POST	`/users/bulk`	admin

Body: `{ users: [{ email, full_name?, role }, ...] }` (1..500 rows). Response: `{ created, skipped, errors, summary }`. Per-row validation; one bad row never aborts the batch. Strictly org-scoped.

Audit export

Method	Path	Auth
GET	`/security-audit-events/export`	admin

Same filters and org-scoping as the audit read endpoint. Returns `text/csv` with a stable column order and a timestamped attachment filename.

Security audit read (🔒 admin only)

Method	Path
GET	`/security-audit-events`

Query: `event_type`, `error_code`, `actor_email`, `q` (substring match against `path` or `detail`), `limit` (1..500, default 50), `offset` (≥0). Response: array ordered newest-first; pagination metadata on `X-Total-Count` / `X-Limit` / `X-Offset` headers. Scoping: rows where `organization_id = caller.org` **OR** `organization_id IS NULL` (pre-auth failures have no caller and are visible to every admin; cross-org denials with an identity stay private to that identity's org). See `23-operator-control-plane.md` for the rationale.

Event validation (phase 12)

`POST /encounters/{id}/events` is now schema-bound:

event_type	required keys
`manual_note`	`note`
`note_draft_requested`	`requested_by`
`note_draft_completed`	`template`
`note_reviewed`	`reviewer`

Server-written types (`encounter_created` , `status_changed`) bypass the validator — they're constructed internally with known-good payloads.

Unknown types → 400 `invalid_event_type` . Non-object or missing-keys `event_data` → 400 `invalid_event_data` .

Pagination (phase 12)

`GET /encounters` accepts `limit` (1..500, default 50) and `offset` (≥0, default 0). Response body is still an array — **backward compatible**.

Totals come on response headers: `X-Total-Count` , `X-Limit` , `X-Offset` .

Error code inventory

Code	HTTP	Origin
`missing_auth_header`	401	auth transport
`unknown_user`	401	auth transport
`auth_mode_unsupported`	500	`CHARTNAV_AUTH_MODE` set to unknown
`role_forbidden`	403	generic `require_roles`
`role_cannot_create_encounter`	403	POST /encounters
`role_cannot_create_event`	403	POST /encounters/{id}/events
`role_cannot_transition`	403	POST /encounters/{id}/status
`cross_org_access_forbidden`	403	body/query asserts other org
`encounter_not_found`	404	missing or cross-org encounter
`location_not_found`	400	bad location_id
`invalid_status`	400	unknown status string
`invalid_initial_status`	400	POST /encounters bad init
`invalid_transition`	400	disallowed state-machine edge

Verification

This full surface is now locked in by two layers:

- pytest (`apps/api/tests/`) — 25 tests, per-test ephemeral SQLite.
- Live smoke (`apps/api/scripts/smoke.sh`) — 9 assertions against a running API.

Both run in CI on every push/PR. See `09-ci-and-deploy-hardening.md` .

Data Model

SQLite (local dev). Schema produced by Alembic migrations

`43ccbf363a8f → a1b2c3d4e5f6` . No schema changes in this RBAC/scoping phase.

Tables (unchanged this phase)

`organizations`

column	type	constraints
id	INTEGER	PK
name	VARCHAR(255)	NOT NULL
slug	VARCHAR(255)	NOT NULL, UNIQUE — **immutable via API**
settings	TEXT	NULL — JSON object, ≤ 16 KB (phase 13, `d4e5f6a7b8c9`). Typed schema as of phase 14 (`OrganizationSettings` — `retention_days` , `default_location_id` , `feature_flags` , `extensions`). Phase 15 wires two `feature_flags` consumers in the frontend — `audit_export` and `bulk_import` (see `25-enterprise-quality-and-compliance.md`).
created_at	DATETIME	NOT NULL default now()

`locations`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
name	VARCHAR	NOT NULL
created_at	DATETIME	NOT NULL default now()

`users`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
email	VARCHAR	NOT NULL, UNIQUE
full_name	VARCHAR	NULL
role	VARCHAR	NOT NULL default `"admin"` , **CHECK (`admin` / `clinician` / `reviewer`)**
is_active	BOOLEAN	NOT NULL default `true`
invited_at	DATETIME	NULL — stamped on admin create (phase 13, `d4e5f6a7b8c9`)
invitation_token_hash	VARCHAR(128)	NULL — sha256 hex (phase 14, `e5f6a7b8c9d0`)
invitation_expires_at	DATETIME	NULL (phase 14)
invitation_accepted_at	DATETIME	NULL (phase 14)
created_at	DATETIME	NOT NULL default now()

Role vocabulary is enforced at BOTH layers since phase 12:

- App: `app/authz.py::KNOWN_ROLES` .
- DB: CHECK constraint installed by migration `c3d4e5f6a7b8` .

Rows with `is_active = 0` are hidden from the default `GET /users` .

Admins can list them with `?include_inactive=1`. Soft-delete preserves FK integrity for audit rows, workflow events, and encounters.

`locations`

Unchanged except for the `is_active` column added in migration `c3d4e5f6a7b8` (same semantics as `users.is_active`).

`encounters` and `workflow_events`

Unchanged. `event_data` is still stored as JSON text; the shape is now gated at the API layer by `EVENT_SCHEMAS` (see `22-admin-governance.md`).

`security_audit_events` (added in phase 10)

column	type	constraints
id	INTEGER	PK
event_type	VARCHAR(100)	NOT NULL, indexed
request_id	VARCHAR(64)	NULL
actor_email	VARCHAR(255)	NULL, indexed
actor_user_id	INTEGER	NULL
organization_id	INTEGER	NULL
path	VARCHAR(512)	NULL
method	VARCHAR(16)	NULL
error_code	VARCHAR(100)	NULL
detail	TEXT	NULL
remote_addr	VARCHAR(64)	NULL
created_at	DATETIME	NOT NULL default now(), indexed

No FKs — audit rows must survive user/org deletion. Populated only on denied or suspicious access (see `18-operational-hardening.md`).

Migration: `b2c3d4e5f6a7`.

Retention (phase 15): `scripts/audit_retention.py` (operator-invoked only) deletes rows older than `CHARTNAV_AUDIT_RETENTION_DAYS` — the app never silently prunes. See `25-enterprise-quality-and-compliance.md` and the retention runbook in `21-staging-runbook.md`.

Seeded tenants (two, with full role coverage)

Org 1 — `demo-eye-clinic` (id=1)

Location: `Main Clinic` (id=1).

email	role
admin@chartnav.local	admin
clin@chartnav.local	clinician
rev@chartnav.local	reviewer

Encounters:

id	patient	provider	status	events
1	PT-1001	Dr. Carter	`in_progress`	3
2	PT-1002	Dr. Patel	`review_needed`	5

Org 2 — `northside-retina` (id=2)

Location: `Northside HQ` (id=2).

email	role
admin@northside.local	admin
clin@northside.local	clinician

Encounters:

id	patient	provider	status	events
3	PT-2001	Dr. Ahmed	`scheduled`	1

Relationships

Unchanged — see `docs/diagrams/er-diagram.md`.

Source of truth per platform mode (phase 16)

Object	`standalone`	`integrated_readthrough` (stub)	`integrated_writethrough` (stub)
organization	ChartNav	mirrored	mirrored
location	ChartNav	mirrored	mirrored
user	ChartNav	ChartNav	ChartNav
encounter	ChartNav	external	external
workflow_event	ChartNav	ChartNav	ChartNav
patient	not supported	external	external
document	ChartNav (as `workflow_events`)	external (read)	external (write via adapter)

Adapter implementations declare their own source-of-truth map in

`AdapterInfo.source_of_truth` — the HTTP surface at `GET /platform`

returns whatever the live adapter says. Frontend surfaces this in the

admin banner. See `26-platform-mode-and-interoperability.md`.

Event taxonomy

event_type	Writer	Shape
`encounter_created`	`POST /encounters` / seed	`{"status": "...", "created_by": "<email>"}`
`status_changed`	`POST /encounters/{id}/status`	`{"old_status": "...", "new_status": "...", "changed_by": "<email>"}`
user-supplied	`POST /encounters/{id}/events`	Any JSON

Build Log

Reverse-chronological.

2026-04-18 — Phase 16: platform mode + interoperability

Step 0 — CI fallout from phase 15 repaired first

- **Postgres parity failure** reproduced locally against a live

Postgres 16. Root cause: migration `c3d4e5f6a7b8` used

`sa.text("1")` as the default for `users.is_active` /

`locations.is_active` (BOOLEAN). SQLite stores booleans as ints

so it accepted the default; Postgres rejects with

`DatatypeMismatch`. Fixed by switching to `sa.text("true")` —

portable across both engines, no new revision required.

- **Frontend CI failure** reproduced in a clean `node:20` Linux

container. Root cause: Vitest 4.1.4 transitively pulled rolldown

+ Vite 8 + esbuild 0.28; the resulting `package-lock.json` was

missing the platform-specific `@esbuild/*` entries npm 10 on

Linux requires (`EBADPLATFORM` / `Missing: … from lock file`).

macOS npm 11 silently tolerated it. Fixed by pinning

`vitest` / `@vitest/ui` to `^3.2.4` (Vitest 3 uses Vite 5 directly,

no rolldown) and regenerating the lockfile. Linux CI replay now

green.

- E2E was only skipped because upstream jobs failed; both fixes

unblock it. Lead-intake / Airtable toast on

`arcgsystems.com/chartnav/ophthalmology` is outside this repo

(zero matches for airtable/webhook/arcgsystems across the repo)

— external Make/Airtable automation owned by the marketing site.

- Head: `aab94c3` after CI fix.

Step 1 — Define platform operating modes

- `app/config.py` gains `platform_mode` and `integration_adapter`

on `Settings`. `CHARTNAV_PLATFORM_MODE` ∈ { `standalone` ,

`integrated_readthrough` , `integrated_writethrough` }. Default

`standalone` . `integrated_*` defaults adapter to `stub` ;

`standalone` pins it to `native` and rejects any other value at

import time.

Step 2 — Adapter boundary

- New package `apps/api/app/integrations/` :

- `base.py` — `ClinicalSystemAdapter` protocol (fetch_patient, search_patients, fetch_encounter, update_encounter_status, write_note, sync_reference_data, `info`), `AdapterInfo`, `SourceOfTruth` enum, `AdapterError` + `AdapterNotSupported`.
- `native.py` — `NativeChartNavAdapter` (persists to ChartNav DB via the same SA Core surface the HTTP routes use; refuses patient ops honestly until a native `patients` table lands).
- `stub.py` — `StubClinicalSystemAdapter(writes_allowed)`. Canned reads; write-through records writes to an in-process list, read-through raises `AdapterNotSupported`.
- `__init__.py` — `resolve_adapter()` + mutable `_VENDOR_ADAPTERS` registry + `register_vendor_adapter(key, factory)`.

Step 3 — HTTP surface

- New `GET /platform` (any authenticated caller). Returns mode + adapter key + display name + description + supports-* flags + source-of-truth map. Zero secret leakage (asserted in tests).

Step 4 — Frontend mode awareness

- `api.ts` gains `PlatformInfo`, `PlatformMode`, `SourceOfTruth` types + `getPlatform(email)` + `platformModeLabel(mode)`.
- `AdminPanel.tsx` fetches `/platform` on mount (alongside `/organization`) and renders a **platform banner** above the tabs: "Platform mode: <mode> · <adapter display name>". Visible on every admin view.
- `styles.css` — new `.platform-banner` rule matching the existing admin look.

Step 5 — Backend tests

- New `tests/test_platform_mode.py` (13 tests): default mode, integrated defaults, invalid mode, standalone-forbids-stub, adapter resolution per mode (native / stub read-through / stub write-through), unknown vendor key, vendor registration path, native refuses unsupported, `/platform` endpoint surface + auth guard. All 131 pytest pass.

Step 6 — Frontend tests

- `AdminPanel.test.tsx` adds 2 tests — banner renders standalone default, banner reflects integrated-readthrough. All mocks updated (`getPlatform` added). Vitest: **30/30 passed**.

Step 7 — Docs

- New `docs/build/26-platform-mode-and-interoperability.md`

(engineering contract).

- New `docs/build/27-adoption-and-implementation-model.md`

(operator/clinic adoption model).

- Updated `01-current-state`, `04-data-model`, `05-build-log`

(this entry), `06-known-gaps`, `08-test-strategy`,

`12-runtime-config`, `15-frontend-integration`,

`16-frontend-test-strategy`.

- `docs/diagrams/system-architecture.md` — added adapter boundary.
- `docs/diagrams/api-data-flow.md` — added adapter resolution

flow.

- `scripts/build_docs.py` picks up sections 26 + 27; executive

summary extended; HTML + PDF regenerated.

Step 8 — Verification

- Backend: **131/131 pytest**, 9/9 smoke via `make verify`.
- Frontend: **30/30 Vitest**, typecheck clean, build emits

~187 KB JS / 8.3 KB CSS.

- Postgres parity: `scripts/pg_verify.sh` — migrate / seed /

smoke / status transition all green against Postgres 16.

- Standalone boot: `CHARTNAV_PLATFORM_MODE=standalone` → native

adapter.

- Integrated boot: `CHARTNAV_PLATFORM_MODE=integrated_readthrough

CHARTNAV_INTEGRATION_ADAPTER=stub` → stub adapter refuses

writes.

2026-04-18 — Phase 15: enterprise quality + compliance signals

Step 1 — Baseline

- Head: `3be3933` (invitations + settings schema + audit export + event hardening + bulk users).
- 110 pytest + 25 Vitest + 12 Playwright + 9 smoke green.

Step 2 — Admin list scaling

- Backend: `GET /users` and `GET /locations` now accept `limit`

(1..500, default 100), `offset` (≥0), `q` substring search, and

`role` (users only). Both endpoints emit `X-Total-Count`, `X-Limit`,

`X-Offset` headers. `include_inactive` continues to work.

- Invalid role filter → 400 `invalid_role`.
- Frontend: `api.ts` gains `listUsersPage` / `listLocationsPage` that

return `{items, total, limit, offset}` by reading the headers.

- `AdminPanel.tsx`: Users + Locations tabs each get a search input +

Prev/Next pager (25/page) + count header. Self-search resets offset

on every change.

Step 3 — Feature-flag consumption

- New `featureEnabled(org, flag)` helper in `api.ts` — flags default

to `true` when unset so the UI doesn't silently strip features for

orgs that haven't touched settings.

- AdminPanel loads `getOrganization(identity)` on mount, holds the

result in state, and passes it into panes that gate UI.

- `audit_export=false` hides the **Export CSV** button. `bulk_import=false`

hides the **Bulk import...** button. Both default-on.

- `flash` in AdminPanel is now `useCallback` -stable, avoiding an

infinite refresh loop that showed up once children started holding

it in `refresh` dependency arrays.

Step 4 — Audit retention helper

- New `apps/api/app/retention.py::prune_audit_events(retention_days, dry_run)` .

App never silently prunes; operators invoke the helper.

- New `CHARTNAV_AUDIT_RETENTION_DAYS` (default 0 = never) in `app/config.py` .
- New `scripts/audit_retention.py` CLI: supports `--days` , `--dry-run` ;

prints a JSON summary.

- New Makefile target `audit-prune ARGS="..."` .

Step 5 — SBOM + image digest

- New `scripts/sbom.py` : captures project + git sha/tag/dirty + image

tag (when set) + `pip list --format json` (API venv) + `npm list

--all --json` `(falls back to` package-lock.json` summary). Honest

`.notes` field calls out that this is not a signed CycloneDX doc.

- `scripts/release_build.sh` now writes `chartnav-sbom-<v>.json` and

`chartnav-api-<v>.digest.txt` (from `docker image inspect`).

- `MANIFEST.txt` sha256s both. `release.yml` attaches both to tag-based

GitHub Releases.

Step 6 — Accessibility baseline

- Installed `@axe-core/playwright` .
- New `apps/web/tests/e2e/a11y.spec.ts` : scans app shell + encounter

list + encounter detail + admin panel (users, audit) + invite

accept. `serious` / `critical` axe findings are blocking.

- Fixes landed while running the baseline:
- `aria-label="Event type"` on the composer `<select>` in App.tsx.
- `aria-label="Role for <email>"` on each inline role `<select>`

in the admin Users table.

Step 7 — Visual regression baseline

- New `apps/web/tests/e2e/visual.spec.ts` : 4 snapshots (encounter list,

admin Users tab, admin Audit tab, invite accept). 1280×820 viewport,

animations disabled via injected stylesheet, `maxDiffPixelRatio: 0.02` .

- Baselines committed for macOS only (`*-chromium-darwin.png`). CI

does NOT run visual — Linux Chromium renders slightly differently.

Honest limitation: documented in `25-enterprise-quality-and-compliance.md`.

- New `e2e-visual` / `e2e-visual-update` Make targets.

Step 8 — CI wiring

- Existing `e2e` job now runs `workflow.spec.ts` + `a11y.spec.ts` (hard gate). Visual is excluded with a comment explaining why.
- Release workflow picks up SBOM + image digest automatically via the updated `scripts/release_build.sh`.

Step 9 — Playwright rate-limit bugfix

- Running the full E2E suite (workflow + a11y + visual) was hitting the rate limiter (`CHARTNAV_RATE_LIMIT_PER_MINUTE=120` default) because all requests come from 127.0.0.1. Fix: set the env to `0` in `playwright.config.ts` 's backend webServer command, which is safe because the E2E DB is always ephemeral.

Step 10 — Backend tests

- New `apps/api/tests/test_enterprise.py` (8 tests): pagination headers + offset + q + role filter + cross-org isolation; role filter 400; retention disabled / dry-run / actual delete; feature flags JSON round-trip.
- Full suite: **118/118 passed**.

Step 11 — Frontend tests

- `AdminPanel.test.tsx` mocks extended for `listUsersPage`, `listLocationsPage`, `getOrganization` feature-flag variants.
- +3 Vitest tests: `audit_export=false` hides export button; `bulk_import=false` hides bulk button; user-search dispatches `listUsersPage({q})`.
- Vitest: **28/28 passed**.

Step 12 — E2E

- a11y: **5/5 passed**.
- Visual: **4/4 passed** against freshly-generated macOS baselines.
- Workflow: **12/12 passed**.
- Total: **21/21 Playwright passed** in ~18s.

Step 13 — Docs

- New `docs/build/25-enterprise-quality-and-compliance.md`.
- Updated `01-current-state`, `05-build-log`, `06-known-gaps`, `03-api-endpoints` (list pagination/search), `04-data-model` (feature_flags consumer note), `08-test-strategy`, `09-ci-and-deploy-hardening` (a11y lane + visual skipped reason), `15-frontend-integration` (search + pager + flag gating), `16-frontend-test-strategy` (a11y + visual), `17-e2e-and-release`

(SBOM + digest bundle), `18-operational-hardening` + `20-observability`

(retention notes), `21-staging-runbook` (retention runbook).

- `scripts/build_docs.py` picks up section 25.
- Final HTML + PDF regenerated.

Step 14 — Hygiene

- Dev DB reset to pristine seeded state before commit.
- Visual baselines committed under `apps/web/tests/e2e/visual.spec.ts-snapshots/` .
- `.gitignore` already excludes caches, `.db` , release dist.

Prior phases

- **Phase 14 — Invitations + schema + audit export + bulk** (`3be3933`)
- **Phase 13 — Operator control plane** (`5a5d846`)
- **Phase 12 — Admin governance** (`4ff4e28`)
- **Phase 11 — Staging deployment + observability** (`ee7cf43`)
- **Phase 10 — Real JWT bearer + operational hardening** (`cbc5184`)
- **Phase 9 — Playwright E2E + release pipeline** (`74fe8dd`)
- **Phase 8 — Create UI + vitest + frontend CI** (`f83d748`)
- **Phase 7 — Frontend workflow UI** (`c4f6e4f`)
- **Phase 6 — Prod auth seam + Docker + Postgres parity** (`700bb0b`)
- **Phase 5 — CI + runtime hardening + doc pipeline** (`cfa8ca9`)
- **Phase 4 — RBAC + full scoping + pytest** (`c6f29e6`)
- **Phase 3 — Dev auth + org scoping** (`efb5b56`)
- **Phase 2 — Strict state machine + filtering** (`505f025`)
- **Phase 1 — Workflow spine** (`93fceb4`)

06 — Known gaps + verification matrix

Known Gaps & Verification Matrix

Verification evidence — phase 14

Local gates

Gate	Result
`make verify` (backend)	✅ **110/110 pytest + 9/9 smoke**
`cd apps/web && npx tsc --noEmit`	✅ clean
`cd apps/web && npx vitest run`	✅ **25/25**
`cd apps/web && npm run build`	✅ 184 KB JS / 8.2 KB CSS
`cd apps/web && npx playwright test`	✅ **12/12** in ~15s
`apps/api/.venv/bin/python scripts/build_docs.py`	✅ HTML + PDF

pytest summary (110)

Suite	Count	Notes
`test_admin.py`	20	admin governance (phase 12)
`test_auth.py`	5	header mode
`test_auth_modes.py`	11	real JWT bearer
`test_control_plane.py`	19	org settings + audit read + invited_at (phase 13 updated for typed settings)
`test_invitations.py` ✦	**20**	**invitations + audit export + event hardening + bulk users (phase 14)**
`test_observability.py`	3	`/ready` , `/metrics`
`test_operational.py`	12	request id, audit, rate limit, CORS
`test_rbac.py`	12	role-gated writes + per-edge transitions
`test_scoping.py`	8	org scoping

Vitest summary (25)

File	Count	Notes
`App.test.tsx`	13	unchanged
`AdminPanel.test.tsx`	12	+3 (invite / bulk summary / audit export) on top of phase 13

Playwright summary (12)

Adds "admin can issue an invitation and download audit CSV" on top of

the 11 scenarios shipped through phase 13.

Phase-16 additions

- **Platform modes**: `CHARTNAV_PLATFORM_MODE` wired — `standalone` / `integrated_readthrough` / `integrated_writethrough` . Config validates and refuses impossible combinations at import time.
- **Adapter boundary**: `app/integrations/` with `ClinicalSystemAdapter` protocol + `NativeChartNavAdapter` + `StubClinicalSystemAdapter` . Vendor adapters plug in via `register_vendor_adapter` .
- **`GET /platform`** : surfaces mode + adapter + source-of-truth. Admin panel renders a mode banner.
- **131 pytest** (+13 platform). **30 Vitest** (+2 platform banner). Playwright unchanged.

- **CI hardening rolled in with this phase**: migration boolean default portability (SQLite → Postgres) + vitest lockfile regen (Linux/Node 20). Both reproduced locally against docker postgres and `node:20` container; both now green.

Phase-15 additions

- **a11y**: 5 axe-core scenarios in CI (`serious` / `critical` blocking). Fixed: event-type `<select>` and inline admin role `<select>` now have aria-labels.
- **Visual regression**: 4 macOS baselines; `make e2e-visual` locally; not in CI (see below).
- **Admin list scaling**: `GET /users` / `/locations` gained `limit` / `offset` / `q` / `role` ; UI adds search + pager.
- **Feature flags**: `audit_export` and `bulk_import` in `feature_flags` actually hide the corresponding admin UI buttons (frontend-tested).
- **Retention**: `scripts/audit_retention.py` + `CHARTNAV_AUDIT_RETENTION_DAYS` env; backend tests cover disabled / dry-run / delete.
- **Release compliance**: `chartnav-sbom-<v>.json` + `chartnav-api-<v>.digest.txt` are produced by `release_build.sh` and attached to the GitHub Release.

Real gaps (prioritized for next phase)

0. **No native `patients` table** — standalone mode's adapter refuses patient operations today. Next standalone-mode build should add the minimum native patient + provider schema.

0. **No real vendor adapter** (Epic / Cerner / Athena / FHIR). The contract + registry exist; vendor work plugs in. Recommended first target: FHIR read-through.

0. **Adapter path isn't fully exercised by HTTP routes yet** — `GET /platform` returns adapter metadata, but encounter/status routes still go through direct DB calls, not the adapter. Standalone that's a nop (native adapter wraps the same DB); for integrated modes this is where the real vendor translation work happens next.

1. **No email delivery** for invitations — admin manually shares the token.

2. **No SSO → users mapping change** (still by `CHARTNAV_JWT_USER_CLAIM`).

3. **Metrics + rate limiter per-process** — multi-worker still needs coordination.

4. **No OpenTelemetry / distributed tracing.**

5. **No log shipping / retention** defined; no audit-table archival.

6. **No CSV export for users or locations** (audit export only).

7. **Forward-only migrations** (acknowledged policy).

8. **No automated staging deploy from CI**.

9. **No signing / SBOM / provenance** on release artifacts.

10. **No JWKS-rotation test, no refresh-token / revocation flow.**

11. **No org-level slug change** (intentional).

12. **No feature-flag consumer yet** — the settings field exists but the app doesn't read it.

13. **No pagination/search on users and locations lists** (fine at current scale).

14. **No visual-regression / a11y audits**.

15. **pytest matrix on Postgres** not wired (fixture env-driven, ready to flip).

16. **No invite-accept screen polish** (redirect to main app on success, proper routing, a11y review) — only the minimal success banner exists.

Auth, Org Scoping, RBAC

1. Authentication (authn)

Source: `apps/api/app/auth.py`.

- Transport today: request header `X-User-Email: <email>`.
- `require_caller` is a FastAPI dependency that resolves the user from

the `users` table and returns a `Caller(user_id, email, full_name, role, organization_id)`.

- Environment flag `CHARTNAV_AUTH_MODE` (default `"header"`) is the

production upgrade seam. When JWT/SSO lands, only the body of

`require_caller` (or a new branch in it) changes. Every route and

every RBAC helper continues to work unchanged because they depend on

`Caller`, not on any header.

Dev-only, explicitly

`X-User-Email` is trivially spoofable. This is acceptable for local

development only. **Do not deploy this to a shared environment

without first swapping the transport.** The seam is there; use it.

Standardized error envelope

Every auth/authz error returns:

```
{"detail": {"error_code": "<stable_code>", "reason": "<human message>"}}
```

Authn error codes

Code	HTTP	When
`missing_auth_header`	401	`X-User-Email` absent or empty.
`unknown_user`	401	Email not found in `users`.
`auth_mode_unsupported`	500	`CHARTNAV_AUTH_MODE` set to something unwired.

2. Authorization (authz / RBAC)

Source: `apps/api/app/authz.py`.

Roles

Role	Intent
`admin`	Full read/write inside own org.
`clinician`	Charting side of the workflow.
`reviewer`	Review side of the workflow; read-only on create/events.

All seeded users today carry exactly one of these roles.

Permission surface

Surface	admin	clinician	reviewer
Read org / locations / users	✓	✓	✓
List / read encounters	✓	✓	✓
Read encounter events	✓	✓	✓
Create encounter	✓	✓	×
Add workflow event	✓	✓	×
Transition scheduled→in_progress	✓	✓	×
Transition in_progress→draft_ready	✓	✓	×
Transition draft_ready→in_progress (rework)	✓	✓	×
Transition draft_ready→review_needed	✓	×	✓
Transition review_needed→draft_ready (kick back)	✓	×	✓
Transition review_needed→completed	✓	×	✓

Authz dependencies

- `require_roles(*roles)` — generic gate, returns `Caller`.
- `require_create_encounter` — applied to `POST /encounters`.
- `require_create_event` — applied to `POST /encounters/{id}/events`.
- `assert_can_transition(caller, from, to)` — called by the status

handler **after** the state machine accepts the edge.

Authz error codes

Code	HTTP	When
`role_forbidden`	403	Generic `require_roles` denies.
`role_cannot_create_encounter`	403	Non-admin/clinician POSTs `/encounters`.
`role_cannot_create_event`	403	Non-admin/clinician POSTs event.
`role_cannot_transition`	403	Role may not drive that specific state edge.

3. Org scoping

Caller's `organization_id` is **authoritative**. Bodies and query

strings cannot override it.

Route	Scoping
`GET /organizations`	`WHERE id = caller.org`
`GET /locations`	`WHERE organization_id = caller.org`
`GET /users`	`WHERE organization_id = caller.org`
`GET /encounters`	`WHERE organization_id = caller.org` + filters
`GET /encounters/{id}`	404 if row is cross-org
`GET /encounters/{id}/events`	404 if parent cross-org
`POST /encounters`	Body `organization_id` must equal caller's; location must belong
`POST /encounters/{id}/events`	404 if cross-org
`POST /encounters/{id}/status`	404 if cross-org

404 vs 403

- **404** when the target might-or-might-not exist in another org —

returning 403 there would leak existence.

- **403** when the client explicitly asserts a different org (body or

`?organization_id=`) — the intent is unambiguous, fail loudly.

4. Diagram

5. Status as of phase 10

- **JWT bearer mode is now implemented** (see `11-production-auth-seam.md`). Real signature + issuer + audience + expiry validation against a JWKS URL.
- **Dedicated audit trail** in `security_audit_events` for denied/suspicious access (see `18-operational-hardening.md`).
- **Rate limiting** on authed paths (see `18-operational-hardening.md`).
- Still TODO: scoped writes to organizations/locations/users, per-tenant key management.

6. What this phase explicitly does NOT do

- No password auth (IdP owns identity proof).
- No refresh-token flow or revocation list.
- No per-tenant JWKS — single issuer + single JWKS URL.

Test & Smoke Strategy

Stack

- `pytest` + FastAPI `TestClient` for unit/integration.
- Per-test temporary SQLite (fresh migrate + seed per test); env-based wiring so the same fixture will later work against Postgres.
- Live shell smoke (`apps/api/scripts/smoke.sh`) used by:
- `make verify` (SQLite)
- `scripts/pg_verify.sh` (Postgres)
- the built Docker image smoke job
- CI runs all of the above on every push / PR.

Layout

```
apps/api/tests/
├── conftest.py          # env-based per-test DB, header presets
├── test_auth.py         # authn surface (5)
├── test_auth_modes.py   # header + bearer seam (3)
├── test_rbac.py         # role-gated writes + state transitions (12)
└── test_scoping.py      # org scoping (8)

apps/api/scripts/
└── smoke.sh             # curl-level live smoke (9 checks)

scripts/
├── verify.sh            # boot + smoke + teardown (SQLite)
└── pg_verify.sh         # same flow against a throwaway Postgres
```

Coverage

Surface	pytest	smoke	Postgres smoke	Docker-image smoke
`/health` , `/`	✓	✓	✓	✓
Header-mode auth	✓	✓	✓	✓
Bearer-mode seam (501 / config)	✓			
Org scoping	✓	✓	✓	✓
RBAC	✓			
Filter on `/encounters`	✓			
State machine invariants	✓		✓	
Event provenance (`changed_by`)	✓		✓	
Migrations			✓	✓
Seed idempotency (x2 runs)			✓	

How to run

```
# Fast pytest (SQLite)
cd apps/api && pytest tests/ -v

# Full local SQLite gate
make verify                # resets DB, pytest, boot, smoke, teardown
```

```
# Postgres parity (docker required)
make pg-verify

# Docker image + smoke
make docker-build
```

Results at time of writing

- `pytest` : **28 passed in ~19s**
- `make verify` (SQLite): 28 pytest + 9 smoke — all green
- `scripts/pg_verify.sh` : migrations + seed (x2) + smoke + status transition + event write — **PASS**

Backend coverage (phase 16)

- **131 pytest.** `test_platform_mode.py` (13) covers:
- Config parsing: default mode, integrated defaults, invalid

mode raises, standalone-forbids-non-native-adapter.

- Adapter resolution: standalone → native, integrated_readthrough

+ stub (writes refused), integrated_writethrough + stub (writes

recorded in-process), unknown vendor key raises, vendor

registration path works.

- Native adapter: honestly refuses patient operations today.
- `GET /platform` endpoint: returns mode + adapter + source-of-truth;

no secret leakage (no `jwt` / `database_url` substrings); requires

auth (401 without identity).

- Frontend: **30 Vitest.** `AdminPanel.test.tsx` adds 2 tests —

platform banner standalone + integrated-readthrough variants.

Backend coverage (phase 15)

- 118 pytest. `test_enterprise.py` (8) covers admin list pagination

(`limit` / `offset` / `X-Total-Count`), `q` substring search on

`/users` + `/locations` , valid + invalid `role` filter on `/users`

(400 `invalid_role`), cross-org isolation of paginated listings,

retention helper (`disabled` / `dry_run` / actual delete paths),

and feature-flag JSON round-trip through `PATCH /organization` .

- Frontend: 28 Vitest. `AdminPanel.test.tsx` adds 3 tests —

`audit_export=false` hides **Export CSV**, `bulk_import=false`

hides **Bulk import...**, search input dispatches

`listUsersPage({q})` .

- E2E: 21 Playwright = 12 workflow + **5 a11y** (axe-core;

`serious` / `critical` blocking) + **4 visual** (local only; macOS

baselines; CI skips due to OS-specific pixel rendering).

Backend coverage (phase 14)

- 110 pytest. `test_invitations.py` (20) covers invite issue / accept happy path / invalid / expired / reused / reissued tokens, cross-org denial, inactive / already-accepted denial, CSV export (admin-only, filters honored, shape), event-hardening rejection cases, and bulk import summary +

org scoping + admin-only.

- `test_control_plane.py` (19) updated for the typed settings schema (extra-forbid, extensions bucket, size cap via extensions).

Backend coverage (phase 13)

- 88 pytest tests. `test_control_plane.py` (17) covers org settings read/patch (all roles + unauth), JSON validation (non-object → 422, oversized → 400 `settings_too_large`), admin PATCH cross-org isolation, audit read admin-only, filters for `event_type` / `actor_email` / `q`, pagination headers, org scoping (never surfaces cross-org rows with identity), and `invited_at` stamping on admin create.

Backend coverage (phase 10)

- 48 pytest tests across `test_auth`, `test_auth_modes`, `test_rbac`, `test_scoping`, `test_operational`.
- Real JWT validation via a locally generated RSA keypair; no external IdP.
- Audit + request-id + rate-limit + CORS all asserted at the integration level.

Frontend verification (phase 8)

- `make web-verify = npm run typecheck + npm test + npm run build`.
- Vitest + Testing Library + jsdom. 12 integration tests mocking

`./api`. Full matrix in `16-frontend-test-strategy.md` and

`06-known-gaps.md`.

- Vite production build still emits `dist/` in ~1s.
- CI now runs a dedicated `frontend` job on every push/PR: node 20,

`npm ci`, typecheck, test, build.

Gaps not yet covered

- pytest matrix against Postgres (the fixture is already env-driven).
- Concurrent writers / race conditions.
- `event_data` schema per `event_type`.
- Pagination behavior (not yet implemented).
- Bearer-mode JWT validation (placeholder only; see `11-production-auth-seam.md`).

CI & Deploy Hardening

GitHub Actions

File: `.github/workflows/ci.yml`. Triggers on **push to `main`** and **every pull request**.

`backend-sqlite`

Install deps → `alembic upgrade head` against an isolated `$RUNNER_TEMP/chartnav_ci.db` via `DATABASE_URL` → seed **twice** (idempotency proof) → `pytest tests/ -v` (28 tests) → `bash scripts/verify.sh` (boots uvicorn, runs `scripts/smoke.sh`).

`backend-postgres` (needs: `backend-sqlite`)

Service container `postgres:16-alpine` on localhost:5432. Install deps (incl. `[postgres]` extra) → `alembic upgrade head` on Postgres → seed twice → boot uvicorn → `scripts/smoke.sh` → live status transition asserts the returned row. Fails the workflow if Postgres parity breaks.

`frontend`

Node 20 + npm cache keyed on `apps/web/package-lock.json` → `npm ci` → `npm run typecheck` → `npm test` (vitest, 12 integration tests) → `npm run build`. Runs in parallel with `backend-sqlite` — the frontend is a peer quality gate, not blocked by backend CI.

`deploy-config`

Independent — runs in parallel with the backend/frontend gates.

- `docker compose config` on all three compose files (dev/staging/prod).
- `shellcheck` on every repo script (`scripts/*.sh`, `apps/api/scripts/smoke.sh`).

A broken compose file or sloppy script lands a red check before anyone runs `staging-up`.

`e2e` (needs: `backend-sqlite`, `frontend`)

Python 3.11 + Node 20 installed. `pip install -e "apps/api[dev,postgres]"` + `npm ci` in `apps/web`. `npx playwright install --with-deps chromium`. Then `npx playwright test tests/e2e/workflow.spec.ts tests/e2e/a11y.spec.ts --reporter=list` — Playwright boots backend on 8001 (SQLite, `CHARTNAV_RATE_LIMIT_PER_MINUTE=0` because all traffic comes from 127.0.0.1) and frontend on 5174, runs the 12 workflow scenarios + **5 axe-core a11y scans** (hard gate — `serious` / `critical` findings fail CI), tears both down. On failure, `playwright-report/` and `test-results/` are uploaded as a workflow artifact for triage.

Visual regression is intentionally excluded from CI (`tests/e2e/visual.spec.ts`, 4 snapshots) — baselines are OS-specific (`*-chromium-darwin.png`) and CI runs on Linux; running visual there would fail first-run. Visual is a local-only gate (`make e2e-visual`). Honest limitation, documented in `25-enterprise-quality-and-compliance.md`.

`release.yml` (separate workflow)

Triggers on `v*.*.*` tag push and manual dispatch. Builds + pushes `ghcr.io/<owner>/chartnav-api:<version>` (`+ :latest`), runs `scripts/release_build.sh` to produce `dist/release/<version>/` (docker-saved image tar, web bundle tar.gz, MANIFEST with sha256s, **SBOM JSON** `chartnav-sbom-<v>.json`, and **image digest** `chartnav-api-<v>.digest.txt` — both added in phase 15), uploads the directory as an artifact, and on tag pushes attaches the files (including SBOM + digest) to a GitHub Release with auto-generated notes. Full reference in `17-e2e-and-release.md`.

`docker-build` (needs: `backend-sqlite`)

Buildx → build `chartnav-api:ci` from `apps/api/` → run the container with `DATABASE_URL=sqlite:///./chartnav.db` and `CHARTNAV_RUN_SEED=1` → poll `/health` → run `scripts/smoke.sh` against the live container. Proves the production image boots end-to-end.

`docs` (needs: `backend-sqlite`)

apt-install Chromium → `python scripts/build_docs.py` with `CHARTNAV_PDF_BROWSER=chromium-browser` → upload HTML + PDF as `chartnav-docs-final` artifact (`if-no-files-found: error`).

What CI catches

Change	Failure surface
Broken migration	alembic step (both DBs)
Broken seed / re-seed collision	seed step (both DBs)
Any pytest regression	`backend-sqlite` pytest
`/health` or auth drift	`scripts/smoke.sh` (all envs)
Cross-org / state-machine regression	smoke + `backend-postgres` transition
Postgres-only SQL dialect break	`backend-postgres`
Image won't build or boot	`docker-build`
Docs pipeline regression	`docs` job

Local verification

```
make verify        # SQLite end-to-end
make pg-verify     # Postgres end-to-end (Docker required)
make docker-build  # build the production image
make docker-up     # full API + Postgres stack (compose)
```

Individual recipes: `install	migrate	seed	test	boot	smoke	docs	reset-db	clean`.

Dev / test / CI / prod DB separation

Surface	Path
Developer local	`apps/api/chartnav.db` (gitignored)
pytest per-test	`tmp_path / "chartnav.db"` (ephemeral)
CI (sqlite)	`$RUNNER_TEMP/chartnav_ci.db` (ephemeral)
CI (postgres)	`postgres:16-alpine` service container
Local pg-verify	throwaway `chartnav-pg-verify` container on port 55432
Prod compose	`chartnav_pgdata` persistent volume

Smoke script (shared contract)

`apps/api/scripts/smoke.sh [BASE_URL]` — 9 curl assertions:

`/health` (code + body), `/me` (401/200), `/encounters` (401/200), cross-org lens 403, own 200, cross-org 404. Exits non-zero on first failure. Used by **every** smoke path (local `make verify` , Postgres parity, Docker image CI).

What this phase adds beyond CI

- Hardened `Dockerfile` (non-root, healthcheck).
- `apps/api/entrypoint.sh` for deterministic migrate-on-start.
- `infra/docker/docker-compose.prod.yml` for the API + Postgres stack.
- `scripts/pg_verify.sh` — reproducible Postgres parity proof.

What this phase does NOT do

- No image push / registry release.
- No Kubernetes / Terraform / cloud-specific bits.
- No TLS termination or edge reverse proxy.
- No secret store integration.

Those land in a dedicated hosted-deploy phase. This phase makes sure

the seams are in place to plug any of them in without rewrites.

Doc Artifact Pipeline

Purpose

The living docs in `docs/build/*` and `docs/diagrams/*` are the source

of truth. The final consolidated artifacts in `docs/final/` are always

rebuildable from those sources — no hand edits.

Source / output layout

```
docs/
├── build/        # human-maintained markdown — source of truth
├── diagrams/     # Mermaid sources (fenced in markdown)
└── final/
    ├── chartnav-workflow-state-machine-build.html   # generated
    └── chartnav-workflow-state-machine-build.pdf    # generated
```

Builder

File: `scripts/build_docs.py`

- Pure Python, no external deps beyond the standard library.
- Resolves paths relative to the script, so the same invocation works

locally and in CI.

- Parses a small markdown subset (headings, fenced code, tables, bullet

lists, inline `code`, **`bold`**) directly to HTML.

- Mermaid fences are passed through verbatim into `<div class="mermaid">`

blocks; the HTML imports Mermaid from a CDN and renders on load.

- Writes `docs/final/chartnav-workflow-state-machine-build.html`.
- Generates the PDF via headless Chromium (`--headless=new --print-to-pdf`).

Selection order:

1. `CHARTNAV_PDF_BROWSER` env var if set (used by CI with `chromium-browser`).

2. macOS Chrome at `/Applications/Google Chrome.app/...`.

3. Chromium in a standard app bundle.

4. `chromium-browser`, `chromium`, `google-chrome`, `chrome` on `PATH`.

- If no browser is available, falls back to `reportlab` for a plain-text

PDF so CI never silently produces a zero-byte artifact.

Run it

Locally:

```
python scripts/build_docs.py          # or:  make docs
```

In CI: the `docs` job in `.github/workflows/ci.yml` installs

`chromium-browser` via apt, runs the builder with `CHARTNAV_PDF_BROWSER=chromium-browser`, and uploads the resulting HTML + PDF as a `chartnav-docs-final` artifact. The upload step is configured with `if-no-files-found: error`, so a silent generation failure fails the build.

Sections consumed (in order)

1. Executive summary (inlined in the builder)
2. `docs/build/01-current-state.md`
3. `docs/build/02-workflow-state-machine.md`
4. `docs/build/03-api-endpoints.md`
5. `docs/build/04-data-model.md`
6. `docs/build/05-build-log.md`
7. `docs/build/06-known-gaps.md`
8. `docs/build/07-auth-and-scoping.md`
9. `docs/build/08-test-strategy.md`
10. `docs/build/09-ci-and-deploy-hardening.md`
11. `docs/build/10-doc-artifact-pipeline.md`
12. `docs/diagrams/system-architecture.md`
13. `docs/diagrams/encounter-status-machine.md`
14. `docs/diagrams/er-diagram.md`
15. `docs/diagrams/api-data-flow.md`

If a source file is missing, the builder emits an inline `*missing: <path>*` marker rather than crashing — a broken diagram or an undeleted section shows up immediately in the rendered output and in the CI artifact.

Why not a heavier stack

- No MkDocs / Docusaurus / Sphinx — those require a theme + build config we'd have to maintain. We currently just need the 10-page PDF to stay fresh.
- No pandoc dependency — keeps the CI image lean and the local experience `python` + a headless browser (which most devs already have).
- If we grow beyond a single consolidated artifact (e.g. a hosted docs site), this builder is easy to retire because the sources are already plain markdown.

11 — Production-auth seam

Production-Auth Seam

State (after phase 10)

Bearer mode is no longer a placeholder — it performs real JWT validation

against a JWKS endpoint. Header mode is preserved for local/dev.

Transports

Mode	State	Behavior
`header`	implemented	Reads `X-User-Email`, resolves from `users`. Dev only.
`bearer`	implemented	Reads `Authorization: Bearer <jwt>`, validates signature/iss/aud/exp against `CHARTNAV_JWT_JWKS_URL`, maps to a `users` row by `CHARTNAV_JWT_USER_CLAIM` (default `email`).

Both produce the same `Caller(user_id, email, full_name, role, organization_id)`.

JWT validation specifics

Implemented in `apps/api/app/auth.py::resolve_caller_from_bearer`:

1. Parse `Authorization: Bearer <token>`. Missing → 401 `missing_auth_header`. Malformed → 401 `invalid_authorization_header`.

2. Look up the signing key via `PyJWKClient(settings.jwt_jwks_url)` (cached). Failure → 401 `invalid_token`.

3. `jwt.decode(...)` with:

- Algorithms: `RS256 / RS384 / RS512 / ES256 / ES384 / ES512`
- `issuer=settings.jwt_issuer`
- `audience=settings.jwt_audience`
- `options={"require": ["exp", "iss", "aud"]}`

4. Specific error codes per failure:

- `token_expired` — `exp` in the past
- `invalid_issuer` — `iss` mismatch
- `invalid_audience` — `aud` mismatch
- `invalid_token` — any other JWT error

5. Pull `settings.jwt_user_claim` (default `email`) out of the claims. Missing / non-string → 401 `missing_user_claim`.

6. `fetch_one` against `users`. Missing → 401 `unknown_user`.

No fallbacks. No "if token looks roughly ok, let it through". Fail closed.

Claim mapping

`CHARTNAV_JWT_USER_CLAIM` (default `email`) names the claim whose value

is used to look up the `users.email` row. In most OIDC setups that's

exactly `email`; for deployments that use a different claim shape, this

is the single knob to turn.

The internal `users` table remains the authoritative source of

`organization_id` and `role`. Tokens carry identity, not authorization.

Test hook

`apps/api/app/auth.py::set_jwk_client(client)` lets tests inject a fake JWKS client. The real `PyJWKClient` is used in production. This is the only crack in the wall that tests are allowed to poke.

Bearer-mode test coverage (`tests/test_auth_modes.py`)

Scenario	Result
Bearer mode without JWT env → `RuntimeError` at import	✅
Header mode default contract unchanged	✅
Valid RS256 token + known user → 200 with role/org	✅
Missing Authorization → 401 `missing_auth_header`	✅
Non-Bearer scheme → 401 `invalid_authorization_header`	✅
Garbage token → 401 `invalid_token`	✅
Wrong issuer → 401 `invalid_issuer`	✅
Wrong audience → 401 `invalid_audience`	✅
Expired token → 401 `token_expired`	✅
Valid token, claim points at unknown user → 401 `unknown_user`	✅
Valid token missing the configured claim → 401 `missing_user_claim`	✅

Tests generate a local RSA keypair and sign tokens with it, then inject a lightweight JWKS stub via `set_jwk_client(...)` so no network calls happen.

What still isn't covered in this phase

- No JWKS rotation test (we cache aggressively via PyJWKClient).
- No HS256 / shared-secret flow — intentionally; production should be asymmetric.
- No refresh-token dance. ChartNav only validates access tokens.
- No revocation list. If a token needs to be killed before `exp`, the identity provider has to roll the signing key (or the user's `users` row can be deleted, which takes the caller down one hop later).

12 — Runtime config

Runtime Configuration

Single source of truth: `apps/api/app/config.py` (`settings`). Every

other module imports from there; nothing else reads `os.environ` .

The contract

Variable	Required?	Default	Purpose
`CHARTNAV_ENV`	no	`dev`	Informational la
`DATABASE_URL`	no (dev default)	`sqlite:///apps/api/chartnav.db`	SQLAlchemy U Postgres.
`CHARTNAV_AUTH_MODE`	no	`header`	`header` (dev) real JWT).
`CHARTNAV_JWT_ISSUER`	**yes iff** `AUTH_MODE=bearer`	—	OIDC issuer UR
`CHARTNAV_JWT_AUDIENCE`	**yes iff** `AUTH_MODE=bearer`	—	Expected `aud`
`CHARTNAV_JWT_JWKS_URL`	**yes iff** `AUTH_MODE=bearer`	—	JWKS endpoin lookup.
`CHARTNAV_JWT_USER_CLAIM`	no	`email`	Token claim us `users.email`
`CHARTNAV_CORS_ALLOW_ORIGINS`	no	`http://localhost:5173,127.0.0.1:5173,localhost:5174,127.0.0.1:5174`	CSV. Empty str only.
`CHARTNAV_RATE_LIMIT_PER_MINUTE`	no	`120`	Per-process sli authed paths.
`CHARTNAV_RUN_SEED`	no	`0`	Entrypoint: run migrations. Ke
`API_HOST` / `API_PORT`	no	`0.0.0.0` / `8000`	Uvicorn bind.
`CHARTNAV_AUDIT_RETENTION_DAYS`	no	`0` (never prune)	Operator-invok (`scripts/auc` Never invoked
`CHARTNAV_PLATFORM_MODE`	no	`standalone`	Operating mod `standalone` `integrated_` `integrated_`
`CHARTNAV_INTEGRATION_ADAPTER`	no	`native` (standalone) / `stub` (integrated)	Selects the ext in integrated m rejects any valu `native` . Venc via `register_ve factory)` in `app/integra`

Validation on startup

`app.config._load()` is evaluated at import time. It enforces:

- `CHARTNAV_AUTH_MODE ∈ {header, bearer}` .
- If `AUTH_MODE=bearer` , **all three** JWT vars must be set — otherwise

a `RuntimeError` is raised listing the missing names. The app will

not boot half-configured.

- `CHARTNAV_PLATFORM_MODE ∈ {standalone, integrated_readthrough,

integrated_writethrough}` — any other value raises at import time.

- `CHARTNAV_PLATFORM_MODE=standalone + CHARTNAV_INTEGRATION_ADAPTER`

anything other than `native` raises at import time (operator error;

native is the only valid adapter in standalone mode).

- `CHARTNAV_RATE_LIMIT_PER_MINUTE` / `CHARTNAV_AUDIT_RETENTION_DAYS`

must be non-negative integers.

Dev / CI / Prod profiles

Profile	How config is supplied	DB	Auth mode	Seed
Dev	`apps/api/.env.example` → `.env` ; Makefile targets	`sqlite:///apps/api/chartnav.db`	`header`	ad-hoc (`make seed`)
Tests	`apps/api/tests/conftest.py` sets env per-test	per-test SQLite in `tmp_path`	`header`	via `scripts_seed.main()`
CI	`.github/workflows/ci.yml` inline env + services	SQLite + live Postgres service	`header`	seed + re-seed in workflow
Prod	platform env / secret store	`postgresql+psycopg://...` via `infra/docker/docker-compose.prod.yml`	`bearer` (when JWT validation is wired)	off by default

Example files

- `apps/api/.env.example` — full dev contract with comments.
- `infra/docker/docker-compose.prod.yml` — wires the API container to

a Postgres service using these env vars; each has a documented

default so the file is runnable as-is for local parity tests.

Database URL specifics

- **SQLite:** `sqlite:///relative/or/absolute/path.db` (three slashes +

absolute path, or three slashes + relative path).

- **Postgres:** `postgresql+psycopg://user:pass@host:5432/dbname` .

The `+psycopg` driver selector is important — SQLAlchemy defaults to

`psycopg2` which is not installed. `psycopg[binary]` is declared in

`apps/api/pyproject.toml` as the `[postgres]` extra.

Staging env contract (phase 11)

`infra/docker/.env.staging.example` captures the contract for the

staging compose file. Every critical value is `${VAR:?}` -guarded in

`infra/docker/docker-compose.staging.yml` so missing env blocks

startup loudly. Extra staging-only vars:

Variable	Purpose
`CHARTNAV_IMAGE_OWNER`	GHCR namespace (e.g. `thekidd2227`).
`CHARTNAV_IMAGE_TAG`	Pinned image tag. Rollback = change this + restart.
`POSTGRES_DB` / `USER` / `PASSWORD` / `PORT`	Required. Never commit real values.

Dev continues to use `apps/api/.env.example`. Runbook: `21-staging-runbook.md`.

Frontend runtime config

The web app reads exactly one env var at build time:

Variable	Required?	Default	Purpose
`VITE_API_URL`	no	`http://localhost:8000`	Base URL the API client points at.

Template: `apps/web/.env.example`. Copy to `.env` to override for a

non-default backend.

Vite substitutes `import.meta.env.VITE_API_URL` into the bundle at build.

Nothing else about the UI comes from env today — dev identity is

selected inside the app and stored in `localStorage`.

What this phase explicitly does NOT do

- No secret store integration (AWS SM, Vault, etc.).
- No `.env` auto-loading in the API (docker-compose and operators load env).
- No per-request feature flags / remote config.

Those are safe to layer on top of this contract without reshaping it.

Deploy Target

Chosen path

Docker image built from `apps/api/Dockerfile`, wired to a Postgres service via `infra/docker/docker-compose.prod.yml`. That's the single supported deploy surface in this phase — no Kubernetes, no Terraform, no cloud-specific glue yet.

Hardened Dockerfile

`apps/api/Dockerfile`:

- `python:3.11-slim` base.
- `PYTHONDONTWRITEBYTECODE`, `PYTHONUNBUFFERED`, disabled pip cache.
- Installs `curl` only (for the container healthcheck).
- Installs the API with the `[postgres]` extra (`psycopg[binary]`).
- Copies `app/`, `alembic/`, `alembic.ini`, `scripts/`, `scripts_seed.py`, `entrypoint.sh`.
- Creates a non-root `chartnav` user (uid 1001) and runs as that user.
- `HEALTHCHECK` hits `/health` every 15s with 3 retries.
- `ENTRYPOINT` is `./entrypoint.sh`; `CMD` is the uvicorn invocation.

Entrypoint

`apps/api/entrypoint.sh`:

1. Asserts `DATABASE_URL` is set (fails loudly otherwise).
2. `alembic upgrade head` — migrations always run at start-up.
3. If `CHARTNAV_RUN_SEED=1`, runs `python scripts_seed.py` (idempotent; off by default in prod).
4. `exec "$@"` — hands off to `CMD` so uvicorn becomes PID 1's child.

Migration-on-start is the right default for this app today. It is idempotent, small, and the seed step is gated off in production.

docker-compose.prod.yml

`infra/docker/docker-compose.prod.yml` brings up two services:

- `db` — `postgres:16-alpine` with a persistent volume, healthcheck, and dev-safe default credentials (operators must override for real deploys).
- `api` — built from `apps/api/Dockerfile`, depends on `db` with `condition: service_healthy`, reads all runtime config from env.

All sensitive values (`POSTGRES_PASSWORD`, JWT settings, etc.) are `${VAR:-default}` so the file is runnable locally for parity tests but ships nothing secret.

Startup path

```
# clone + cd
cd infra/docker
# optional: override defaults via env
docker compose -f docker-compose.prod.yml up --build
```

Boot sequence per container:

1. Postgres starts, becomes healthy (~5s).

2. API container starts, entrypoint runs migrations.

3. uvicorn binds 0.0.0.0:8000.

4. HEALTHCHECK turns green.

Runtime config path

Every variable is documented in `12-runtime-config.md` . The

image respects only those — there's no bespoke `config.yaml` ,

Consul, etc. to reason about.

CI build verification

`.github/workflows/ci.yml` → `docker-build` job:

1. Buildx sets up.

2. Builds `chartnav-api:ci` from `apps/api/` .

3. Runs the container with `DATABASE_URL=sqlite:///./chartnav.db` and

`CHARTNAV_RUN_SEED=1` .

4. Polls `/health` .

5. Runs the same `scripts/smoke.sh` the backend-sqlite job uses.

6. Tears down.

If the image can't boot, serve `/health` , or pass smoke, the job fails.

Staging deployment (phase 11 overlay)

Staging rides on the same Dockerfile + entrypoint. The dedicated

surface lives in `infra/docker/docker-compose.staging.yml` +

`infra/docker/.env.staging.example` , driven by

`scripts/staging_{up,verify,rollback}.sh` . Full reference:

`19-staging-deployment.md` and the operator runbook

`21-staging-runbook.md` .

Key differences from this phase's generic prod compose:

- Image is **pinned** to `CHARTNAV_IMAGE_TAG` — rollback = repin + restart.
- API healthcheck uses `/ready` (DB-aware) rather than `/health` .
- Ports bind to `127.0.0.1` only; a reverse proxy is expected to front it.
- Every critical env var is `${VAR:?msg}` -guarded so a missing value

blocks startup with an explicit error.

Out of scope this phase

- Image registry push (no target credentials by default).
- Kubernetes manifests, Helm charts, Terraform.
- TLS termination — expected to be handled by a reverse proxy in front.
- Secret management integration (AWS SM, Vault, etc.).
- Blue/green or rolling deployment orchestration.

Those are the next "hosted deploy" phase's job. The Docker seam is

real and ready to be plugged into any of them.

Postgres Parity

Why this phase

The app was born on SQLite. This phase proves the same code runs on

Postgres — migrations, seed, auth, scoping, RBAC, state machine.

Code-level changes that made this possible

The DB layer was rewritten to use **SQLAlchemy Core** (`apps/api/app/db.py`).

That single switch buys cross-dialect behavior:

- All SQL uses **named** bind parameters (`:name`). No more `?` .
- Portable functions only — `COALESCE` replaced `IFNULL` ; `CURRENT_TIMESTAMP` already worked on both.
- `insert_returning_id(conn, table, values)` helper encapsulates the

dialect split: `RETURNING id` on Postgres, `cursor.lastrowid` on

SQLite. Callers never touch either.

- `transaction()` context manager gives a SA connection with `begin()`

semantics on either backend.

Consumers changed:

- `apps/api/app/api/routes.py` — all queries now use `:name` binds

and `transaction()` .

- `apps/api/scripts_seed.py` — rewritten to `transaction()` +

`insert_returning_id` , no more raw `sqlite3` .

- `apps/api/app/auth.py` — uses `fetch_one` from the new `db` module.
- `apps/api/alembic/env.py` — honors both `-x sqlalchemy.url=` and

`DATABASE_URL` env, so migrations can run against either backend.

No existing migration (`43ccbf363a8f` , `a1b2c3d4e5f6`) needed changes.

They already used SQLAlchemy constructs and portable defaults.

Proof — `scripts/pg_verify.sh`

Single reproducible command. Launches `postgres:16-alpine` in a

throwaway container, runs migrations + seed, boots the API against it,

runs the full smoke, then does a live **status transition** + reads

the resulting `workflow_events` row back.

Local run on 2026-04-18:

```
==> alembic upgrade head on Postgres
INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO  [alembic.runtime.migration] Running upgrade  -> 43ccbf363a8f, create initial chartnav tables
INFO  [alembic.runtime.migration] Running upgrade 43ccbf363a8f -> a1b2c3d4e5f6, add encounters and workflow_events

==> seed (idempotent, run twice)
Seed complete.
  demo-eye-clinic: organization_id=1 location_id=1
  northside-retina: organization_id=2 location_id=2
```

```
==> smoke: http://127.0.0.1:8766
  ok  /health 200
  ok  /health body.status=ok
  ok  /me 401 without auth
  ok  /me 200 role=admin org=1
  ok  /encounters 401 without auth
  ok  /encounters 200 admin1
  ok  /encounters?organization_id=2 (cross-org) -> 403
  ok  GET /encounters/1 as admin1 -> 200
  ok  GET /encounters/3 cross-org -> 404
==> all smoke checks passed
==> extra: status transition on Postgres
  ok  PT-1001 in_progress -> draft_ready (clinician)
  ok  workflow_events row recorded on Postgres
==> Postgres parity: PASS
```

CI coverage

`.github/workflows/ci.yml` → `backend-postgres` job:

1. Spins up a `postgres:16-alpine` service container on port 5432.

2. Installs the API with the `[postgres]` extra.

3. `alembic upgrade head` (direct env `DATABASE_URL=postgresql+psycopg://…`).

4. Seeds twice (idempotency proof).

5. Boots uvicorn, waits for `/health` .

6. Runs `scripts/smoke.sh` .

7. Executes the same `clinician → draft_ready` status transition and

asserts the returned row.

If any step fails, the job fails and nothing merges.

What's covered / not covered

Surface	Verified on Postgres
Schema migrations	✅
Idempotent seed	✅
App boot	✅
`/health` , `/me`	✅
`/encounters` (list + filters + cross-org 403)	✅
`/encounters/{id}` 200 / 404 cross-org	✅
State transition write + event emit	✅
pytest suite (28 tests)	Runs on SQLite only today — see Known Gaps.

The pytest suite still targets SQLite because the TestClient fixture

uses `tmp_path` file DBs. Migrating that to a Postgres matrix is the

obvious next CI upgrade; the DB layer is ready for it.

Frontend Integration

Goal

Turn the backend workflow slice into a usable app. An operator should

be able to identify themselves, filter their org's encounters, open one,

see its timeline, append events, and drive role-appropriate status

transitions — all through the UI — without the backend contract changing.

Stack

- Vite 5 + React 18 + TypeScript (already in the repo).
- No component library yet — one small CSS file (`styles.css`) + inline

attributes. Easy to swap later if the UI grows.

- No state library — `useState` / `useEffect` around a typed API module.

Source layout

```
apps/web/src/
├── api.ts            # typed API client (single fetch surface)
├── identity.ts       # dev identity helpers (seeded list + localStorage)
├── App.tsx           # shell, list, detail, timeline, actions
├── main.tsx          # React entrypoint, imports styles.css
├── styles.css        # one CSS file (CSS variables; no build config)
└── vite-env.d.ts     # Vite ambient types
```

`api.ts` — contract

- `API_URL` comes from `VITE_API_URL` (falls back to `http://localhost:8000`).
- Every function takes an `email: string` ; the client sets

`X-User-Email` on each request. When the backend moves to bearer

mode, only this module changes.

- `ApiError(status, errorCode, reason)` — every non-2xx response is

converted to this, so UI code can render the backend's

`{error_code, reason}` envelope directly.

- Exposed functions (match the backend 1:1):
- `getHealth()`
- `getMe(email)`
- `listEncounters(email, filters)`
- `getEncounter(email, id)`
- `getEncounterEvents(email, id)`
- `createEncounterEvent(email, id, {event_type, event_data})`
- `updateEncounterStatus(email, id, status)`
- `listLocations(email)`
- Pure helpers used for UI affordances only:
- `allowedNextStatuses(role, currentStatus)` — mirrors

`authz.TRANSITION_ROLES` . Drives which transition buttons appear.

- `canCreateEvent(role)` — drives the "Add event" composer.
- **Backend remains the source of truth** — the UI never assumes the server will accept anything; any 4xx surfaces as a banner with the error code + reason.

Dev identity

- `apps/web/src/identity.ts` ships the seeded demo users:
- Org 1: `admin@` , `clin@` , `rev@chartnav.local`
- Org 2: `admin@` , `clin@northside.local`
- Selector in the header lets operators switch caller without editing code. "Custom email…" lets a real/unseeded email be entered for testing 401 paths.
- Selection persists in `localStorage.chartnav.devIdentity` .

UI layers

Header

- Brand wordmark.
- Chip showing the resolved caller (`email · role · org N`) — or the auth error if `/me` failed.
- Chip with the current API base URL.
- Identity picker.

Layout (two-column, collapses below 920px)

- **Encounter list (left)** — filter bar (`status` , `provider_name` , `location_id`) + row list with patient id, name, provider, and a color-coded status pill.
- **Detail pane (right)** — headline + status pill, facts grid (organization / location / scheduled / started / completed / created), allowed-transition buttons, timeline of events, and "Add event" composer when the current role is permitted to write.

Role-aware actions

- Allowed transitions are shown as buttons. If none are available for the (role, current_status) pair, we show a note instead of fake disabled buttons.
- The "Add event" composer is hidden for reviewers (matches `CAN_CREATE_EVENT`). Reviewers still get a subtle note explaining why.
- After any successful action the UI refreshes: detail, events, and list.
- After a failure, the UI shows the exact `error_code + reason` from the backend in a top banner.

UX states

State	Surface
Loading	"Loading…" in list / detail
Empty list	"No encounters match these filters."
401 / unknown	Red chip in the header: `auth: 401 unknown_user — ...`
403 role	Banner: `403 role_cannot_transition — ...`
404 cross-org	Banner: `404 encounter_not_found — ...`
400 bad status	Banner: `400 invalid_transition — ... allowed next ...`

Local dev

```
# one terminal — backend
make install          # venv + pip install -e "apps/api[dev,postgres]"
make reset-db         # fresh SQLite + seed
make boot             # uvicorn on 8000

# second terminal — frontend
make web-install      # npm install
make web-dev          # vite on 5173

# or both at once (Ctrl-C to stop both)
make dev
```

`VITE_API_URL` in `apps/web/.env.example` defaults to

`http://localhost:8000`, matching `make boot`. Copy to `.env` if you

need to point at a different backend.

Build / verification

```
make web-typecheck    # tsc --noEmit
make web-build        # vite build -> apps/web/dist
```

Verified manually on 2026-04-18:

Check	Result
`tsc --noEmit`	✅
`npm run build` emits dist/	✅ (154 KB JS / 6 KB CSS)
`GET /me` for all 5 seeded users	✅
`GET /encounters` org-scoped (org1 → 2 rows; org2 → `[3]`)	✅
`GET /encounters?status=in_progress`	✅ `['PT-1001']`
`GET /encounters/{id}/events` hydrated	✅ 3 events
Clinician in_progress → draft_ready	✅ 200
Clinician review_needed → completed (denied)	✅ 403 `role_cannot_transition`
Reviewer review_needed → completed	✅ 200
Reviewer POST event (denied)	✅ 403 `role_cannot_create_event`
Admin POST event	✅ 201
Switching identity re-fetches `/me` and list	✅

Encounter creation (phase 8)

- Header shows `+ New encounter` for admin + clinician; hidden for reviewer.

- `CreateEncounterModal` fetches `/locations` (server-scoped to caller org), auto-selects when only one option exists, and submits `{ organization_id, location_id, patient_identifier, patient_name, provider_name, status }`.
- Required fields (`patient_identifier` , `provider_name` , `location_id`) block submit until filled.
- While in-flight the submit button disables and shows `Creating…` .
- On success the list refreshes, the new encounter auto-selects, and a green banner shows `#<id> created` .
- On failure the modal stays open and the exact `{error_code, reason}` appears inline, so reviewers can correct and retry.

UX hardening (phase 8)

- Every mutating control (transition, append event, create) disables while its request is in flight and shows a pending label.
- Identity badge has explicit states (`identity-loading` , `identity-error` , `identity-badge`) so loading vs. failed vs. resolved are distinguishable.
- Banners use ARIA `role="alert" | "status"` and `data-testid` so assistive tech + tests can target them.
- When no transition is legal for the current `(role, status)` pair, we show a plain note — no fake-disabled buttons.

Full-stack E2E (phase 9)

Playwright boots both backend (SQLite + seeded) and frontend together,

and runs 8 Chromium scenarios that exercise the real UI end-to-end —

identity resolution, scope switching, encounter create, event append,

role-aware transitions, reviewer restrictions, unknown-email auth

surface, filters. Details in `17-e2e-and-release.md` . Command:

`make e2e` .

Admin governance (phase 12)

- **Admin button** in the header renders only when `isAdmin(role)` . Clinician / reviewer never see it.
- Opens `<AdminPanel />` — a modal with **Users** and **Locations** tabs.
- Users: create form (email / full name / role), table with inline role change + deactivate/reactivate. Self-row is disabled so admins can't lock themselves out.
- Locations: create form + inline rename (click-to-edit) + deactivate.
- Every mutating control disables in flight and surfaces backend `{error_code, reason}` in a per-tab banner on failure.
- The event composer's event_type is now a `<select>` wired to the backend allowlist (`EVENT_TYPES`), so the UI can't submit invalid types.
- Encounter list paginates 25 rows at a time via `listEncountersPage` , rendering Prev/Next + "N-M of T" when `total > 25` .

Operator control plane (phase 13)

- Admin panel is now **4 tabs**: Users, Locations, **Organization**, **Audit log**.
- Organization tab: readonly `slug` , editable `name` + free-form `settings` JSON (16 KB cap enforced server-side). Local JSON parse errors surface before any PATCH is fired.
- Audit log tab: filter row (`event_type` , `actor_email` , free-text `q`), paginated table (25/page) ordered newest-first with timestamp / event / actor / method+path / error code / request id columns. Backend 4xx surfaces as banner.
- Users tab renders an "Invited" badge for active users with `invited_at` set.

Invitations + bulk + audit export (phase 14)

- Users tab now has an **Invite** button per active non-self row; clicking it surfaces the raw token in a one-shot banner (never shown again, dismissible).
- Users tab also has a **Bulk import...** button that opens a CSV-style textarea dialog; the submit result renders an inline created/skipped/errors summary before closing.
- Organization tab replaced the raw JSON textarea with typed inputs for `name` , `default_provider_name` , `encounter_page_size` , `audit_page_size` , plus an **Extensions** JSON textarea for forward-compat drift.
- Audit log tab has an **Export CSV** button that fetches with the auth header and triggers a local download (must use fetch + blob — a raw `<a href>` can't send `X-User-Email`).

- New minimal **Invite accept** screen (`apps/web/src/InviteAccept.tsx`) mounted at `/invite?invite=<token>` and `/accept` . On success stores the email into `localStorage.chartnav.devIdentity` so the main app picks it up for header-mode dev use.

Admin list scaling + feature flags + a11y (phase 15)

- Users + Locations tabs now have a **search input** + **Prev/Next pager** (25/page) + count header, powered by `listUsersPage` / `listLocationsPage` which read `X-Total-Count` / `X-Limit` / `X-Offset` headers and return `{items, total, limit, offset}` . Self-search on either tab resets `offset` to 0 so pagination never strands the user mid-query. Users tab also supports a `role` filter.
- `featureEnabled(org, flag)` is a new helper in `api.ts` — flags default to `true` when unset so untouched orgs keep full UI. `AdminPanel` loads `getOrganization(identity)` on mount and threads the org object into panes that gate UI. `feature_flags.audit_export=false` hides the **Export CSV** button on the Audit tab; `feature_flags.bulk_import=false` hides the **Bulk import...** button on the Users tab. Server is unchanged — this is a UX toggle, not a security control.
- `flash` in `AdminPanel` is now a stable `useCallback` — child panes that put it in a `refresh` dep array no longer hit an infinite refresh loop. (Regression landed while wiring feature-flag reloads; fix validated by Vitest + E2E.)
- Accessibility: event-type `<select>` in the encounter detail composer now carries `aria-label="Event type"` ; inline role `<select>` s in the admin Users table carry `aria-label="Role for <email>"` . These were the only `serious` / `critical` axe findings — the suite is now clean.

Platform mode awareness (phase 16)

- `api.ts` gains `getPlatform(email)` + `PlatformInfo` /

`PlatformMode` / `SourceOfTruth` types + `platformModeLabel`

helper.

- `AdminPanel` calls `/platform` on mount (alongside

`/organization`) and renders a **platform banner** above the

tabs: `Platform mode: <label> · <adapter display name>` . Visible

on every admin view so the deployment's semantics are never

implicit.

- Two Vitest scenarios (`src/test/AdminPanel.test.tsx`):
- Standalone default renders "ChartNav native" adapter label.
- Integrated read-through renders the stub adapter label +

"Integrated — read-through" mode text.

- No UI changes for clinicians — the banner is admin-only, inside

the admin modal.

What this phase explicitly does NOT do

- No real login flow — `X-User-Email` is still dev transport.
- No global state manager (Redux/Zustand) — state lives in `App.tsx` .
- No UI component library.
- No pagination (the backend doesn't paginate yet).
- No optimistic updates — every mutation re-fetches the relevant slice.

Those are future phases. The API client + identity seam make them

cheap to add without rewriting.

Frontend Test Strategy

Stack

- **Vitest** (`vitest` + `@vitest/ui`) — same Vite runtime as the app.
- **jsdom** — DOM environment.
- **@testing-library/react** + **@testing-library/user-event** — user-centric queries, realistic events.
- **@testing-library/jest-dom** — richer DOM matchers.

Layout

```
apps/web/
├── vite.config.ts            # vitest config block (jsdom, globals, setup file)
├── src/test/
│   ├── setup.ts              # imports jest-dom matchers, cleans DOM + localStorage between tests
│   └── App.test.tsx          # 12 integration tests against the mocked API layer
└── package.json              # scripts.test = vitest run, test:watch = vitest
```

Mocking strategy

`vi.mock("../api", ...)` replaces every backend call with a `vi.fn()`

while preserving the pure helpers (`allowedNextStatuses` , `ApiError` ,

etc.). That keeps tests decoupled from the network, deterministic, and

fast — but still exercises the real `App.tsx` component tree.

The mocks emulate the server's real error envelope so the UI code's

`{error_code, reason}` branch is actually tested.

Coverage (12 tests)

Scenario	Check
`/me` resolves → identity badge shows email/role/org	✅
Mocked list renders both seeded encounters	✅
Changing `status` filter calls `listEncounters` with filter arg, list updates	✅
Selecting an encounter loads detail + timeline; status shown	✅
Clinician sees only operational transition buttons (`draft_ready`)	✅
Reviewer sees review-stage transitions (`completed` , `draft_ready`)	✅
Reviewer sees no event composer, replaced by `event-denied` note	✅
Reviewer cannot see the `+ New encounter` button	✅
Admin: open create modal, submit, success banner, modal closes	✅
Create failure: backend 403 `cross_org_access_forbidden` surfaces in modal error, modal stays open	✅
Switching identity refetches `/me` and the list (org1 → org2)	✅
Unknown-user identity: custom email path surfaces the 401 chip	✅
Status transition call path refreshes detail + events, success banner	✅

Run

```
# one-shot (CI + local gate)
cd apps/web && npm test

# watch mode
cd apps/web && npm run test:watch

# combined frontend gate
make web-verify      # typecheck + test + build
```

Result at time of writing: **12 passed in ~2.5s**.

Philosophy

- The UI is tested the way a user drives it — click, select, type — not

via internal implementation details.

- We assert the UI **reflects the server's authority**: when the backend

returns a 4xx envelope, the UI must show the `error_code` and

`reason` verbatim, not invent its own copy.

- Role-aware affordances are tested by identity switching in the

picker, not by poking at internal state.

End-to-end tests (Playwright)

Full browser coverage ships in phase 9 — see `17-e2e-and-release.md` .

Playwright boots real backend + frontend, runs 8 scenarios in Chromium,

and tears both servers down cleanly. Command: `make e2e` .

Vitest and Playwright are fully isolated: Vitest's `test.include` is

narrowed to `src/**/*.test.{ts,tsx}` and excludes `tests/**` , so the

two suites never cross-contaminate.

Invitations + bulk + export tests (phase 14)

- `AdminPanel.test.tsx` :
- Invite button on a user row issues a token and surfaces it in `admin-invite-token` .
- Bulk import summary renders created/skipped/errors counts.
- Audit Export CSV button wires `downloadAuditExport` with current filters.
- Playwright: 1 new end-to-end scenario — admin creates a user, clicks **Invite**, token banner visible; Audit tab → **Export CSV** triggers a real browser download with the expected filename pattern.
- Backend's 20 `test_invitations.py` tests are the source of truth for server behavior; the UI tests verify wiring and user-visible states.

Operator control-plane tests (phase 13)

- `AdminPanel.test.tsx` — 4 new tests: Organization tab loads + PATCH dispatch, local JSON parse error path, Audit tab row render + filter dispatch, Audit 403 surfaces as error banner.
- Playwright — 1 new scenario: admin opens Organization tab → edits name → saves → opens Audit tab → audit table + filter UI render.

Admin governance tests (phase 12)

- `src/test/AdminPanel.test.tsx` — 5 Vitest tests: user list, create user success + 409 error, self-row disabled, location create on the Locations tab.
- `src/test/App.test.tsx` — added 1 test asserting the Admin button is visible to admins only.

- E2E `workflow.spec.ts` — 2 new scenarios: admin creates a user + a location end-to-end; clinician never sees the Admin button.

Enterprise quality tests (phase 15)

- `AdminPanel.test.tsx` extends to **12 tests** (+3). New mocks:

`listUsersPage` , `listLocationsPage` , `getOrganization` , `inviteUser` ,

`bulkCreateUsers` , `downloadAuditExport` — all required because

AdminPanel now holds feature-flag-aware org state and calls the

paginated list endpoints.

- `audit_export=false` → `admin-audit-export` absent, `admin-audit-refresh` still present.
- `bulk_import=false` → `admin-user-bulk-open` absent.
- Users-tab search input dispatches `listUsersPage({ q })` .
- Playwright grows to **21 tests**:
- `workflow.spec.ts` — 12 (unchanged).
- `a11y.spec.ts` — **5 axe-core scans** across app shell, encounter

list, encounter detail, admin Users tab, admin Audit tab, invite

accept. `serious` / `critical` findings are blocking. Fixes landed

with the baseline: `aria-label="Event type"` on the composer

`<select>` ; `aria-label="Role for <email>"` on each inline role

`<select>` .

- `visual.spec.ts` — **4 Playwright screenshots** (encounter list,

admin Users, admin Audit, invite accept). Viewport 1280×820,

animations disabled via injected stylesheet,

`maxDiffPixelRatio: 0.02` . Baselines are OS-specific and shipped

for macOS only (`*-chromium-darwin.png`). **Local-only** gate:

`make e2e-visual` / `make e2e-visual-update` . CI skips visual

because Linux Chromium renders slightly differently.

- Rate limiter: Playwright's backend webServer sets

`CHARTNAV_RATE_LIMIT_PER_MINUTE=0` — the full suite (workflow +

a11y + visual) runs many requests from 127.0.0.1 and would

otherwise hit the 120/min limit. Safe because the E2E DB is

always ephemeral.

Platform mode tests (phase 16)

- `AdminPanel.test.tsx` extends to **17 tests** (+2).
- New mocks: `getPlatform` . The default `beforeEach` seeds a

standalone-native platform response; individual tests override

to integrated-readthrough.

- Assertions:
- `admin-platform-banner` renders on every admin view.
- `admin-platform-mode` content changes with

`platform_mode` — standalone vs integrated_readthrough.

- `admin-platform-adapter` content matches the adapter's

`display_name` .

Gaps not yet covered

- Visual regression not in CI (documented; OS-specific baselines).
- No keyboard-only / screen-reader manual QA pass beyond axe's

automated ruleset.

- The create modal doesn't exercise location-list errors yet.
- Loading skeletons / spinner content not asserted.

E2E & Release

E2E harness

Stack

- **Playwright** (`@playwright/test`) with the `chromium` browser.
- Playwright's `webServer` boots **both** backend and frontend, waits for

their `/health` / `/` URLs, and tears both down on exit. No bash

orchestration, no orphan processes.

Config

`apps/web/playwright.config.ts` :

- **Backend** boots on `127.0.0.1:8001` using an ephemeral SQLite file

at `apps/api/.e2e.chartnav.db` (the operator's dev DB is never

touched). The config runs:

```
rm -f .e2e.chartnav.db

alembic upgrade head

python scripts_seed.py

uvicorn app.main:app --port 8001
```

- **Frontend** boots on `127.0.0.1:5174` via `npm run dev -- --host 127.0.0.1 --port 5174` , with `VITE_API_URL` pointed at the E2E backend.
- `reuseExistingServer: !CI` — locally, running `make e2e` twice in a

row reuses the stack; in CI each run is fresh.

- Traces, screenshots, videos retained on failure only.

Vitest / Playwright isolation

`vite.config.ts` now narrows Vitest `test.include` to `src/**/*.test.{ts,tsx}` and excludes `tests/**` , so Playwright specs and Vitest specs never cross-contaminate.

E2E coverage

`apps/web/tests/e2e/workflow.spec.ts` (8 tests):

Scenario
App boots, resolves default seeded identity
Switching identity (admin1 → admin2) swaps visible encounter scope
Admin opens detail, creates encounter, sees it in the list
Admin appends an event; it appears in the timeline
Clinician performs operational transitions; reviewer edge not offered
Reviewer sees completion/kick-back controls, no create button, no event composer
Unknown email surfaces `identity-error` chip with `unknown_user`
Status filter narrows the list

Tests use resilient selectors: `role`, placeholder text, and `data-testid` for elements without another stable hook.

Local run:

```
make e2e          # headless
make e2e-headed   # visible browser
make e2e-ui       # Playwright UI mode
```

Result at time of writing: **8/8 passed in ~14s**.

CI

New `e2e` job in `.github/workflows/ci.yml`:

1. `backend-sqlite` + `frontend` must pass first.

2. Set up Python 3.11 + Node 20 (both with caches).

3. `pip install -e "apps/api[dev,postgres]"` (system install — verify.sh already tolerates missing `.venv`).

4. `npm ci` in `apps/web`.

5. `npx playwright install --with-deps chromium`.

6. `npx playwright test --reporter=list` (Playwright boots + tears down the stack).

7. On failure, upload `playwright-report/` + `test-results/` as an artifact for debugging.

Release pipeline

`scripts/release_build.sh`

One command to produce a releasable bundle under `dist/release/<version>/`:

- `chartnav-api-<version>.tar` — `docker save` of `chartnav-api:<version>` (importable with `docker load`).
- `chartnav-web-<version>.tar.gz` — static bundle from `apps/web/dist` (drop behind any CDN/reverse proxy).
- `MANIFEST.txt` — version, git sha, ref, build time, artifact sizes + sha256s.

Version resolution:

1. Explicit arg (e.g. `bash scripts/release_build.sh v0.1.0`).

2. Exact git tag on `HEAD`.

3. Fallback `dev-<short-sha>`.

Local:

```
make release-build VERSION=v0.1.0
```

`dist/release/` is gitignored.

`.github/workflows/release.yml`

Triggers:

- `push` on tags matching `v*.*.*` (e.g. `v0.1.0`, `v1.2.3-rc1`).
- `workflow_dispatch` with a `version` input (manual).

Steps:

1. Resolve version from the tag or input.

2. Install `apps/web` deps (Node 20).

3. Login to `ghcr.io`.

4. `docker buildx build + push` of `ghcr.io/<owner>/chartnav-api:<version>` **and** `:latest`.

5. Run `scripts/release_build.sh <version>` to produce the local artifact set.

6. Upload the whole `dist/release/<version>/` as a workflow artifact (always).

7. On tag pushes only, create a **GitHub Release** with:

- auto-generated notes from the commit log
- `chartnav-api-<version>.tar`, `chartnav-web-<version>.tar.gz`, `MANIFEST.txt` attached.

Permissions: `contents: write` for the release, `packages: write` for `ghcr.io`.

Release is never auto-triggered from CI commits — only an explicit tag push or manual dispatch cuts one.

Release flow (operator view)

```
# 1. work lands on main, CI green
git checkout main && git pull --ff-only

# 2. tag
git tag -a v0.1.0 -m "ChartNav v0.1.0"
git push origin v0.1.0

# 3. GitHub Actions runs the release workflow, pushes the GHCR image,
#    creates the Release, attaches the tarballs.

# 4. deploy
docker pull ghcr.io/<owner>/chartnav-api:v0.1.0
docker compose -f infra/docker/docker-compose.prod.yml up -d
```

Staging artifact (phase 11)

`scripts/release_build.sh` now also writes `chartnav-staging-<version>.tar.gz`

containing the staging compose file, its env template, and the runbook

scripts (`staging_up.sh`, `staging_verify.sh`, `staging_rollback.sh`)

plus the three new doc pages (19/20/21). `release.yml` attaches the

tarball to the GitHub Release alongside the API image tar and web

bundle, so a staging operator can `curl` one archive and be ready to

run against the pinned image tag.

SBOM + image digest (phase 15)

`scripts/release_build.sh` now emits **two additional files** per

release:

- `chartnav-sbom-<version>.json` — produced by `scripts/sbom.py`.

Captures project + version, git sha/ref/tag/dirty, image owner +

tag (when `CHARTNAV_IMAGE_TAG` is set), full `pip list --format

json` `(API venv), full` npm list --all --json `tree of` apps/web`

(falls back to `package-lock.json` summary if `node_modules` isn't

installed). The `.notes` field is explicit that this is a plain

JSON inventory — **not** a signed CycloneDX document. Upgrading to

`cyclonedx-py` + `cyclonedx-npm` is the obvious next step.

- `chartnav-api-<version>.digest.txt` — `docker image inspect

--format '{{.Id}}'` of the release image. Deployers can diff the

digest of what they pulled from GHCR against what was released.

Both files are sha256'd in `MANIFEST.txt` (the manifest is still the

integrity anchor) and both are attached to the GitHub Release by

`release.yml` . Full reference in

`25-enterprise-quality-and-compliance.md` .

E2E coverage (phase 15 addendum)

Playwright now runs 21 tests across 3 spec files:

- `workflow.spec.ts` — 12 (unchanged contract).
- `a11y.spec.ts` — 5 axe-core scans (hard gate — `serious` / `critical`

findings fail CI).

- `visual.spec.ts` — 4 Playwright screenshots. **Local only**:

baselines are macOS-specific (`*-chromium-darwin.png`); CI runs on

Linux and would fail first-run. Documented honestly.

`playwright.config.ts` 's backend webServer sets

`CHARTNAV_RATE_LIMIT_PER_MINUTE=0` so the full suite doesn't trip

the per-IP rate limiter when every request comes from 127.0.0.1.

What this phase does NOT do

- No staging / production deploy target is connected — the release workflow builds and publishes, operators still run `docker compose` wherever they host it.
- No automated Postgres E2E — Playwright's webServer uses SQLite for speed + isolation. Postgres parity is still proven by `backend-postgres` .
- No rollback automation — rollbacks are `docker pull` of the previous tag.
- No cosign / Sigstore signing, no CycloneDX / SPDX, no SLSA provenance. The SBOM is the input to those next steps.

18 — Operational hardening

Operational Hardening

Landing this phase gives ChartNav four production-baseline primitives

that every long-running service needs: **request correlation**,

structured logs, **audit trail for denied access**, and **abuse

protection**.

1. Request correlation

Middleware: `apps/api/app/middleware.py::RequestIdMiddleware`.

- Every request gets a `request_id` on `request.state`.
- Honors an inbound `X-Request-ID` if it's a reasonable length (≤ 64 chars); else generates a fresh UUID4 hex.
- Always echoed back as `X-Request-ID` on the response — including error responses (the exception handler copies it through).

2. Structured logging

Module: `apps/api/app/logging_config.py`.

- JSON-per-line to stdout via `JsonFormatter`.
- Fields: `timestamp` (UTC ISO8601), `level`, `logger`, `message`, plus any `extra={}` the caller passes.
- `AccessLogMiddleware` emits one line per request with `request_id`, `method`, `path`, `status`, `duration_ms`, `user_email` (from resolved caller), `organization_id`, `remote_addr`.
- The HTTP exception handler emits a `WARNING auth_denied` line for every denied auth/scoping response, attaching `error_code` and caller context.
- Uvicorn's access logger is quieted to avoid duplicating our access log.

Log sample (pretty-printed here for readability):

```
{"timestamp":"2026-04-18T05:35:45Z","level":"INFO","logger":"chartnav.http",
 "message":"request","request_id":"494e2888ab414831805bf79cdefa7fc2",
 "method":"GET","path":"/me","status":401,"duration_ms":3.55,
 "user_email":null,"organization_id":null,"remote_addr":"testclient"}
```

3. Security audit trail

Module: `apps/api/app/audit.py`.

Migration: `b2c3d4e5f6a7 — add security_audit_events`.

Schema

Table `security_audit_events`:

- `id`, `event_type`, `request_id`, `actor_email`, `actor_user_id`, `organization_id`, `path`, `method`, `error_code`, `detail`, `remote_addr`, `created_at`.
- Indexed on `event_type`, `actor_email`, `created_at`.

What gets audited

The HTTP exception handler (`apps/api/app/main.py::_http_exception_handler`) writes an audit row when:

- The response is **401** or **403**, or
- The response `error_code` is in the `AUDITED_ERROR_CODES` set:

`missing_auth_header`, `unknown_user`, `invalid_authorization_header`, `invalid_token`, `token_expired`, `invalid_issuer`, `invalid_audience`, `missing_user_claim`, `cross_org_access_forbidden`, `role_forbidden`, `role_cannot_create_encounter`,

`role_cannot_create_event`, `role_cannot_transition`, `rate_limited`.

What is NOT audited

- No raw JWT or `Authorization` header value.
- No password material (there isn't any, but explicitly).
- No successful requests — those belong in the access log.

Never-throw guarantee

`audit.record(...)` swallows its own exceptions. A broken audit insert cannot mask the underlying 4xx the user actually hit.

4. CORS

- `allow_origins=["*"]` is gone.
- Driven by `CHARTNAV_CORS_ALLOW_ORIGINS` (comma-separated). Default covers Vite dev on `:5173` + E2E on `:5174`.
- In production, set it explicitly to the real frontend origin(s).
- Empty string → same-origin only.
- `allow_credentials=true`; `allow_headers` is limited to `Authorization`, `Content-Type`, `X-User-Email`, `X-Request-ID`; `expose_headers` exposes `X-Request-ID` so browsers can surface it for debugging.

5. Rate limiting

Middleware: `apps/api/app/middleware.py::RateLimitMiddleware`.

- In-memory sliding window (60s), per-process.
- Keyed on `(client_ip, path)`.
- `CHARTNAV_RATE_LIMIT_PER_MINUTE` (default 120). `0` disables.
- Protects `/me`, `/encounters`, `/organizations`, `/locations`, `/users` (and descendants). `/health` and `/` are never rate-limited.
- On limit exceed: `HTTP 429` with standardized envelope `{error_code: "rate_limited", reason: ...}` AND an audit row.
- Honest limitation: **per-process**, not distributed. Multi-worker / multi-node deployments should sit behind an edge limiter that coordinates globally. Documented in `12-runtime-config.md`.

Config reference

Env var	Default
`CHARTNAV_CORS_ALLOW_ORIGINS`	`http://localhost:5173,http://127.0.0.1:5173,http://localhost:5174,http://127.0.0.1:5174`
`CHARTNAV_RATE_LIMIT_PER_MINUTE`	`120`

See `12-runtime-config.md` for the full contract.

Test coverage

`apps/api/tests/test_operational.py` (12 tests):

Scenario	Result
Inbound `X-Request-ID` round-trips	✅
Missing `X-Request-ID` → server generates one	✅
Error responses also carry `X-Request-ID`	✅
Audit row on `missing_auth_header`	✅
Audit row on `unknown_user`	✅
Audit row on `cross_org_access_forbidden` (query lens)	✅
Audit row on `role_cannot_create_encounter`	✅
No audit row on successful requests	✅
Rate limit returns **`429 rate_limited`** with envelope	✅

Scenario	Result
Rate limit disabled when `CHARTNAV_RATE_LIMIT_PER_MINUTE=0`	✅
CORS preflight allows a configured origin	✅
CORS preflight rejects an unconfigured origin	✅

Observability (phase 11 additions)

This baseline now includes a live observability surface — see

`20-observability.md` :

- `GET /ready` — DB-aware readiness. Backs the staging container `HEALTHCHECK` .
- `GET /metrics` — Prometheus text exposition. Counters cover request traffic, auth denials (by `error_code`), rate-limited responses, audit event writes (by `event_type`), and request latency sum/count.
- Middleware + exception handler now feed those counters as part of normal request handling. `audit.record` also bumps `chartnav_audit_events_total{event_type}` before the DB insert.

Audit read API (phase 13)

`GET /security-audit-events` (admin only) surfaces the audit table

in-product — org-scoped per `23-operator-control-plane.md` , filterable

by `event_type` , `error_code` , `actor_email` , `q` (substring),

paginated via `limit` / `offset` + `X-Total-Count` headers, ordered

newest-first. Never exposes raw tokens or secrets.

Audit export (phase 14)

`GET /security-audit-events/export` — admin only, CSV, honors the

same filters as the read endpoint. Operator use case: spot-check or

ship into a longer-lived SIEM. No column contains secrets or raw

tokens by construction.

Audit retention helper (phase 15)

Rather than silently pruning in-app (which hides data loss behind a

side effect), ChartNav ships an **operator-invoked** helper:

- `apps/api/app/retention.py::prune_audit_events(retention_days, dry_run)`

deletes `security_audit_events` rows older than `retention_days` .

The app never calls this from a request path; it runs only when an

operator invokes the CLI.

- `scripts/audit_retention.py` — `--days` , `--dry-run` ; JSON summary

(`status` , `retention_days` , `cutoff` , `matched` , `deleted` ,

`dry_run`). Also callable via `make audit-prune ARGS="..."` .

- `CHARTNAV_AUDIT_RETENTION_DAYS` (default `0`) supplies the

threshold; `0` disables the helper even if invoked.

Pairing: a typical deployment runs this nightly via cron against the

API pod's venv. Runbook notes live in `21-staging-runbook.md` ;

compliance framing in `25-enterprise-quality-and-compliance.md` .

Admin list scaling (phase 15)

`GET /users` and `GET /locations` gained `limit` / `offset` / `q` (+ `role`

on users) query parameters with `X-Total-Count` / `X-Limit` / `X-Offset`

response headers. Invalid `role` → 400 `invalid_role`. `include_inactive`

still works. Defaults remain backward-compatible (no params →

first 100 rows).

Remaining gaps

- Rate limiter is in-memory per process — same caveat applies to the new metrics.
- Distributed tracing (OpenTelemetry) not wired yet.
- No log shipping / retention policy defined.
- Audit retention helper exists, but no in-app scheduler and no

archival-to-S3 flow — operator cron is the contract.

- No dashboards/alerts are shipped — `20-observability.md` documents the alert points, wiring is the operator's responsibility.

Staging Deployment

Shape

ChartNav's staging deploy unit is a **pinned Docker image** (`ghcr.io/<owner>/chartnav-api:<tag>`) plus a `postgres:16-alpine` container, wired together by `infra/docker/docker-compose.staging.yml` . That's the whole substrate. There is no Kubernetes, no Terraform, no serverless. Rolling out = change a tag + restart. Rolling back = change the tag back + restart.

This is intentional. ChartNav is still a small service; a boring, reproducible compose deploy is the honest size. Phase-out-to-something-bigger will be a deliberate future phase, not an accident.

Artifacts added this phase

Path	Purpose
`infra/docker/docker-compose.staging.yml`	API + Postgres, pinned image, healthcheck on `/ready` , volumes, restart policy.
`infra/docker/.env.staging.example`	The env contract. Copy to `.env.staging` and fill in.
`scripts/staging_up.sh`	`docker compose config` -validate then `up -d` .
`scripts/staging_verify.sh`	Smoke + observability check against the running stack.
`scripts/staging_rollback.sh <prev_tag>`	Repin `CHARTNAV_IMAGE_TAG` , pull, restart API, wait for `/ready` .

Topology

```
┌─────────────────┐        ┌──────────────────────┐
│  reverse proxy  │──TLS──▶  api:8000 (127.0.0.1)  │
│  (ops layer)    │        │  ghcr.io/..../chartnav  │
└─────────────────┘        │  -api:${TAG}           │
                           │  entrypoint: migrate   │
                           │  + uvicorn             │
                           └──────────────────────┘
                                      │
                                      │ postgresql+psycopg
                                      ▼
                            ┌────────────────────┐
                            │ db:5432            │
                            │ postgres:16-alpine │
                            │ volume:            │
                            │  chartnav_staging_ │
                            │  pgdata            │
                            └────────────────────┘
```

- API and DB both bind to `127.0.0.1` on the host — reverse proxy terminates TLS and handles public routing.
- Healthcheck runs `curl -fsS http://127.0.0.1:8000/ready` inside the container: because `/ready` touches the DB, that one call also verifies migrations ran and the DB connection is live.
- DB persistence: named volume `chartnav_staging_pgdata` . Volumes survive `docker compose down` ; they do **not** survive `docker compose down -v` . A rollback to a prior image tag never needs to touch the volume.

Migration-on-start

`apps/api/entrypoint.sh` runs `alembic upgrade head` before it execs `uvicorn` . Staging respects that, so "deploy = new image" and "migrate" are the same action. Two properties fall out for free:

- Deploys are idempotent: apply the same tag twice, get the same schema.
- Downgrade paths are an image concern, not an ops concern: redeploying an older image with its own Alembic head will only try to apply migrations it knows about.

If a migration ever needs to run offline (big data reshape), that's the one thing this flow doesn't help with — pin the old image, take an outage window, run `alembic` manually, then deploy the new image.

Required env contract (summary)

Full reference: `12-runtime-config.md`. Staging adds nothing new; it just requires real values for what was optional in dev:

Var	Notes
`CHARTNAV_IMAGE_OWNER` / `_TAG`	GHCR owner + pinned tag.
`POSTGRES_DB/USER/PASSWORD`	Never commit real values.
`CHARTNAV_AUTH_MODE`	`bearer` in real staging. `header` only for first-touch QA.
`CHARTNAV_JWT_*`	Required when `AUTH_MODE=bearer`; the app refuses to import otherwise.
`CHARTNAV_CORS_ALLOW_ORIGINS`	**Never `*`**. Real staging frontend origin(s) only.
`CHARTNAV_RATE_LIMIT_PER_MINUTE`	Defaults to 120; bump for load-test windows.

Compose uses `${VAR:?message}` for the critical vars so a missing value blocks `docker compose up` with an explicit error.

Commands

```
# first-time: copy + fill in
cp infra/docker/.env.staging.example infra/docker/.env.staging
# edit infra/docker/.env.staging with real values

# deploy
make staging-up          # docker compose config + up -d
make staging-verify      # smoke + observability checks (see 21)
# real traffic

# rollback to a previously-published tag
make staging-rollback TAG=v0.0.9
make staging-verify

# teardown (keeps the DB volume)
make staging-down
# nuke DB too
cd infra/docker && docker compose -f docker-compose.staging.yml --env-file .env.staging down -v
```

Rollback — honest description

"Rollback" in this deploy is:

1. `scripts/staging_rollback.sh <prev_tag>` rewrites `CHARTNAV_IMAGE_TAG` in `.env.staging`.
2. `docker compose pull api` fetches the prior image from GHCR.
3. `docker compose up -d api` restarts only the API service; Postgres is untouched.
4. The script polls `/ready` and fails loudly if it doesn't come back green within ~40s.

What rollback **cannot** do on this substrate:

- Undo a destructive migration. If revision N dropped a column, going back to N-1's image won't restore the column. Treat destructive migrations as one-way on this deploy.
- Blue/green. There's one API replica.
- In-flight request draining. `docker compose up -d` is SIGTERM-then-wait. If you need graceful draining of long operations, put them behind a queue, not the web process.

Those are fair tradeoffs for a compose-based staging. The migration-on-start path also means the rollback script doesn't need to run Alembic by hand — the old image knows its own head.

CI validation

`.github/workflows/ci.yml` → `deploy-config` job: on every push/PR it runs `docker compose … config` on all three compose files (dev/staging/prod) and `shellcheck` on every repo script. A broken compose file or sloppy script lands a failing check before the operator ever tries to `staging-up`.

Observability

ChartNav's observability baseline is honest: three surfaces (`/health` , `/ready` , `/metrics`), one structured log stream, and one audit table. No platform fantasy.

Surfaces

`GET /health` — liveness

Cheap. Never touches the DB. Use it for load-balancer keepalives where "the process is answering" is enough.

```
200 {"status":"ok"}
```

`GET /ready` — readiness

Runs `SELECT 1` against the database. Use it for orchestrator healthchecks where you want to gate traffic on DB reachability. `503 not_ready` if the DB is unreachable.

```
200 {"status":"ready","database":"ok"}
```

In `infra/docker/docker-compose.staging.yml` , the API container's `HEALTHCHECK` targets `/ready` — not `/health` — because a running uvicorn with a broken DB connection is worse than no uvicorn.

`GET /metrics` — Prometheus text exposition

Unauthed. Exposed on the internal network only; restrict at the edge if you must expose publicly (see below).

Counters emitted:

Metric	What it answers
`chartnav_requests_total{method,path,status}`	Traffic by (method, path, status bucket `1xx..5xx`).
`chartnav_auth_denied_total{error_code}`	Denied auth/scoping attempts, keyed on the stable error codes from the `{error_code, reason}` envelope.
`chartnav_rate_limited_total`	`429 rate_limited` responses.
`chartnav_audit_events_total{event_type}`	Audit rows written, by event type.
`chartnav_http_request_duration_ms_sum`	Cumulative request latency, milliseconds.
`chartnav_http_request_duration_ms_count`	Sample count matching the sum. Divide for a mean.

Honest caveats:

- **In-process, single-worker.** Same caveat as the rate limiter. Multi-worker uvicorn = each worker scrapes its own counters. A proper metrics lib backed by Prometheus multiprocess mode is the next step if that becomes a real problem.
- No histograms / percentiles. `sum + count` is the honest minimum; percentiles lie when they're synthesized from one bucket.

Structured logs

All logs are line-delimited JSON on stdout. Ship them with whatever collector the deploy target uses (compose log driver, cloudwatch agent, vector, fluent-bit — the app doesn't care).

Access log (INFO, `logger=chartnav.http`)

Emitted once per request:

```
{
```

```
  "timestamp": "2026-04-18T15:22:10Z",
  "level": "INFO",
  "logger": "chartnav.http",
  "message": "request",
  "request_id": "a1b2c3...",
  "method": "POST",
  "path": "/encounters/1/status",
  "status": 200,
  "duration_ms": 7.42,
  "user_email": "clin@chartnav.local",
  "organization_id": 1,
  "remote_addr": "10.0.0.42"
}
```

Auth-denied log (WARNING, `logger=chartnav`)

Emitted alongside every audited denial — mirror image of the audit row:

```
{
  "timestamp": "2026-04-18T15:22:10Z",
  "level": "WARNING",
  "logger": "chartnav",
  "message": "auth_denied",
  "request_id": "a1b2c3...",
  "path": "/me",
  "method": "GET",
  "status": 401,
  "error_code": "missing_auth_header",
  "user_email": null,
  "organization_id": null
}
```

Request failure log (ERROR)

Uncaught exceptions surface with a stack trace via the access-log middleware. Always carries `request_id`.

Fields guaranteed across all records

- `timestamp` — UTC ISO8601
- `level` — standard logging level
- `logger`
- `message`
- `request_id` — when the log is request-scoped

Collection notes

- Docker compose: default stdout goes to the `json-file` driver unless you override `logging.driver` on the service. For staging, point at your collector (`loki`, `fluent-bit`, etc.) via a compose override file; the app emits valid JSON already.
- Do not parse with `grep` in production. Ship to a log store and query by `request_id`, `error_code`, or `user_email`.

Audit table (`security_audit_events`)

See `18-operational-hardening.md` for the schema. The audit table is a durable peer to the logs: logs answer "what happened and when", the audit table answers "what denied-access attempts, who did them, to which org, how often". They're not redundant — logs churn, audit rows persist.

Query examples:

```
-- recent denials by error_code
SELECT error_code, COUNT(*) AS n
FROM security_audit_events
WHERE created_at > now() - interval '24 hours'
GROUP BY error_code
ORDER BY n DESC;

-- repeated cross-org attempts from one actor
SELECT actor_email, path, COUNT(*) AS n
FROM security_audit_events
WHERE event_type = 'cross_org_access_forbidden'
```

```
  AND created_at > now() - interval '7 days'
GROUP BY actor_email, path
HAVING COUNT(*) >= 3;
```

What to check during staging smoke

The one-shot is `make staging-verify` (see `21-staging-runbook.md`). Interactively, an operator should look at:

1. **Liveness**: `curl -fsS http://.../health` → `200`.

2. **Readiness**: `curl -fsS http://.../ready` → `200 database=ok`. If this fails, API logs will explain; `docker compose logs db` will confirm.

3. **Request id round-trip**: `curl -I -H "X-Request-ID: probe-$$" http://.../health` — response must carry the same id back.

4. **Auth surface**:

- `curl -i http://.../me` without auth → 401 + a `missing_auth_header` entry on the next `/metrics` scrape.
- `GET /metrics` contains `chartnav_auth_denied_total{error_code="missing_auth_header"}`.

5. **Rate-limit sanity**: quickly spam `GET /me` to exceed the configured `CHARTNAV_RATE_LIMIT_PER_MINUTE`; expect a 429 and `chartnav_rate_limited_total` to tick up.

6. **Audit row check**: `docker compose exec -T db psql -U $POSTGRES_USER -d $POSTGRES_DB -c "SELECT event_type, COUNT(*) FROM security_audit_events GROUP BY 1 ORDER BY 2 DESC;"`.

Alert points (contract, not implementation)

When an operator wires alerting, these are the knobs that carry real meaning:

Signal	Meaning
`/ready` failing for >1 minute	DB connection lost or pool exhausted.
`chartnav_auth_denied_total{error_code="invalid_token"}` burst	IdP trouble or attack traffic hitting `Authorization:`.
`chartnav_auth_denied_total{error_code="cross_org_access_forbidden"}` spike	Misconfigured client, token reuse across orgs, or probing.
`chartnav_rate_limited_total` steep climb	Abuse or a stuck client.
Audit table growth rate ≫ steady-state	Investigate; correlate with the two metrics above via request_id.

This phase does not ship dashboards or alert rules. The plumbing is in place; wire them in your observability platform of choice (Grafana/Datadog/CloudWatch/etc.).

Audit retention (phase 15)

The app no longer has to silently prune old audit rows to stay bounded.

Operators run `scripts/audit_retention.py` (or `make audit-prune`)

against the API venv; the script honors `CHARTNAV_AUDIT_RETENTION_DAYS`

(default `0` = never prune) and can be invoked with `--dry-run` for a

report-only pass. JSON output covers `matched` / `deleted` / `cutoff`.

A nightly cron is the expected wiring. Full framing in

`25-enterprise-quality-and-compliance.md`; cron example in

`21-staging-runbook.md`.

This keeps two concerns explicit: (1) the app never changes audit

state outside of a request-scoped write path, and (2)

compliance-facing retention lives in ops with a scripted contract

instead of hidden behavior.

Remaining gaps

- No tracing / span propagation yet (OpenTelemetry is the natural next layer).
- Metrics are per-process.
- Retention is handled by an operator-invoked helper, archival-to-S3 and SIEM shipping are still ops infra's job.

21 — Staging runbook

Staging Runbook

Operator-grade playbook. Every command is real and reproducible. No magic.

---

0. Prerequisites

- Docker + `docker compose` v2.
- Access to `ghcr.io/<owner>/chartnav-api` (at minimum anonymous pull; the image is published public on release).
- A JWKS URL from your IdP (for bearer mode).
- Ability to place DNS / reverse-proxy config in front of port 8000 on the staging host. The compose file binds to `127.0.0.1:8000` — your reverse proxy is expected to handle public exposure and TLS.

1. First-time provisioning

```
# on the staging host
git clone https://github.com/thekidd2227/chartnav-platform.git
cd chartnav-platform
cp infra/docker/.env.staging.example infra/docker/.env.staging
# edit .env.staging with real values — see docs/build/19-staging-deployment.md
```

Validate before starting anything:

```
docker compose \
  --env-file infra/docker/.env.staging \
  -f infra/docker/docker-compose.staging.yml \
  config >/dev/null
```

Failure here (`required variable X is missing`) is good: the compose file intentionally `${VAR:?}` -blocks on every critical var.

2. Deploy

```
make staging-up          # == bash scripts/staging_up.sh
make staging-verify      # == bash scripts/staging_verify.sh
```

Expected finish:

```
==> staging verify: PASS
```

If verify fails at `/ready` , the API can't reach the DB. Check in order:

1. `docker compose -f infra/docker/docker-compose.staging.yml ps` — db should be `healthy` .
2. `docker compose -f infra/docker/docker-compose.staging.yml logs --tail=100 db` .
3. `docker compose -f infra/docker/docker-compose.staging.yml logs --tail=100 api` .

If verify fails at the auth-denial metrics check, the app is up but the exception handler didn't record the denial — check the API container's logs for a Python traceback.

3. Exercising bearer mode

`staging_verify.sh` only drives `/me` + `/encounters` as an authed caller when `CHARTNAV_AUTH_MODE=header` . In bearer mode the script stops after unauth surfaces + observability checks, because it cannot mint tokens.

Manual path:

1. Pull a real JWT from your IdP (e.g. `auth0` CLI, `okta` CLI, a curl against the IdP's token endpoint).

2. Confirm the token's `email` claim (or whatever you set `CHARTNAV_JWT_USER_CLAIM` to) matches a row in `users`.

3. Run:

```bash

curl -sS -H "Authorization: Bearer $TOKEN" http://127.0.0.1:8000/me | jq

curl -sS -H "Authorization: Bearer $TOKEN" http://127.0.0.1:8000/encounters | jq 'length'

```

4. Watch `docker compose logs -f api` in another terminal; the access log should show `user_email` and `organization_id`.

If bearer validation fails, the `error_code` in the 401 body pinpoints the reason — `invalid_token`, `invalid_issuer`, `invalid_audience`, `token_expired`, `missing_user_claim`, `unknown_user`. Every one of those also writes a `security_audit_events` row.

4. Rollback

The contract: "image tag rollback restores behavior, not data."

```
# Pin a previously-published tag and restart the API only.
make staging-rollback TAG=v0.0.9
make staging-verify
```

What the script does (honest):

1. Rewrites `CHARTNAV_IMAGE_TAG=` in `infra/docker/.env.staging`.

2. `docker compose pull api`.

3. `docker compose up -d api`.

4. Polls `/ready` for ≤40s.

What it does **not** do:

- Reverse any migrations. If release `N` applied a destructive migration (dropped a column, dropped a table), going back to `N-1`'s image will **not** restore data. Forward-only migrations are the policy — plan accordingly before tagging a release.
- Blue/green. There's one API replica. During the restart, `/ready` will go red for a handful of seconds; the reverse proxy should drain accordingly.

If rollback fails `/ready`, you're worse off than before rollback and need to triage:

1. `docker compose logs --tail=200 api`.

2. Pin the **current broken** tag back (`make staging-rollback TAG=<the tag you started from>`) — sometimes the issue is "the image didn't actually pull".

3. If the DB is broken (not the image), the recovery path is a DB restore from your backup. That is out-of-band of this repo; document it in your infra runbook.

5. Health / observability quick checks

```
# liveness + readiness
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/ready

# metrics snapshot
curl -fsS http://127.0.0.1:8000/metrics | grep -E '^(chartnav_requests_total|chartnav_auth_denied_total|chartnav_rate_limited_

# recent audit rows (inside the db container)
docker compose -f infra/docker/docker-compose.staging.yml \
  --env-file infra/docker/.env.staging \
  exec -T db psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" \
  -c "SELECT id, event_type, actor_email, path, error_code, created_at \
      FROM security_audit_events ORDER BY id DESC LIMIT 20;"

# follow logs (JSON; pipe through jq for readability)
```

```
docker compose -f infra/docker/docker-compose.staging.yml \
  --env-file infra/docker/.env.staging \
  logs -f api | sed 's/^[^{]*//' | jq -cR 'fromjson? | select(.) | .'
```

See `20-observability.md` for the full field reference.

6. Teardown

```
# soft — keep the DB volume
make staging-down

# hard — DROP ALL DATA
cd infra/docker && docker compose \
  --env-file .env.staging -f docker-compose.staging.yml down -v
```

Never run the hard teardown on a shared staging host without confirming with whoever else uses it.

7. Common failures, honest fixes

Symptom	Likely cause	Fix
`required variable CHARTNAV_JWT_JWKS_URL is missing`	`CHARTNAV_AUTH_MODE=bearer` but JWT config blank in `.env.staging`	Fill in the three JWT vars or set `AUTH_MODE=header` for triage.
`db` never becomes healthy	Volume permissions / old volume with wrong password	`docker volume rm docker_chartnav_staging_pgdata` **iff** you accept data loss.
Smoke passes but app UX is broken	Stale frontend bundle vs. backend contract	Re-pin frontend asset (out of scope of this compose) — the API is fine.
429 everywhere	`CHARTNAV_RATE_LIMIT_PER_MINUTE` too low for a load test	Temporarily raise; remember it's per-process, per-path, per-IP.
`invalid_issuer` / `invalid_audience` flood	Wrong `CHARTNAV_JWT_ISSUER` or `_AUDIENCE` vs. the IdP's tokens	Inspect a sample token's `iss` / `aud`; align env.

8. Audit retention cron (phase 15)

ChartNav never silently prunes audit rows. Retention is explicit,

operator-invoked, and scripted:

```
# dry-run against current env default
make audit-prune ARGS="--dry-run"

# explicit threshold, report-only
python scripts/audit_retention.py --days 90 --dry-run

# actually delete
python scripts/audit_retention.py --days 90
```

The helper reads `CHARTNAV_AUDIT_RETENTION_DAYS` (default `0` =

never) when `--days` is omitted. Output is JSON: `status`,

`retention_days`, `cutoff`, `matched`, `deleted`, `dry_run`.

A typical cron hook on the staging host:

```
# every day at 02:30 UTC, prune anything older than 90 days
30 2 * * *  cd /srv/chartnav && \
  apps/api/.venv/bin/python scripts/audit_retention.py --days 90 \
    >> /var/log/chartnav/audit-prune.jsonl 2>&1
```

Check the last run by tailing the log or by inspecting the

`security_audit_events` row count. Full framing in

`25-enterprise-quality-and-compliance.md` and `20-observability.md` .

9. What this runbook intentionally does not cover

- Backup & restore. That's your infra concern; the app is stateless beyond Postgres.
- DNS / TLS / edge routing.
- Disaster recovery beyond image rollback.
- Secret rotation. Rotate in your secret store, then `make staging-up` re-reads env on container restart.

When any of these become first-class ChartNav concerns, they graduate to their own build-log phase and doc section.

Admin Governance

Phase 12 adds real administrative control on top of the RBAC + observability stack: admins can manage users and locations for their org, the DB enforces role vocabulary, workflow events are schema-bound, and the encounter list paginates. No more read-only metadata; no more free-form event drift.

1. DB-level role constraint

Migration `c3d4e5f6a7b8` adds a CHECK constraint:

```
CHECK (role IN ('admin', 'clinician', 'reviewer'))
```

SQLite's `batch_alter_table` rebuilds the `users` table to install it; Postgres picks it up as a normal `ADD CONSTRAINT`. `app.authz.KNOWN_ROLES` matches exactly. Inserts with any other role now bounce at the `IntegrityError` layer, before the app even sees them.

The same migration adds `is_active BOOLEAN NOT NULL DEFAULT true` to `users` and `locations` so metadata can be soft-deleted without breaking FK integrity on historical encounters, workflow events, or audit rows.

2. Admin CRUD API

All admin endpoints require role `admin` via `authz.require_admin`. Any other role gets `403 role_admin_required`. All operations are strictly scoped to the caller's org.

Users

Method	Path	Body / Query	Errors
GET	`/users`	`?include_inactive=1`	—
POST	`/users`	`{email, full_name?, role}`	`invalid_role` (400), `user_email_taken` (409), `role_admin_required` (403)
PATCH	`/users/{id}`	`{email?, full_name?, role?, is_active?}`	`invalid_role`, `user_email_taken`, `cannot_demote_self`, `cannot_deactivate_self`, `user_not_found` (cross-org)
DELETE	`/users/{id}`	soft-delete (sets `is_active=0`)	`cannot_deactivate_self`, `user_not_found`

Locations

Method	Path	Body / Query	Errors
GET	`/locations`	`?include_inactive=1`	—
POST	`/locations`	`{name}`	`role_admin_required`
PATCH	`/locations/{id}`	`{name?, is_active?}`	`location_not_found` (cross-org)

Method	Path	Body / Query	Errors
DELETE	`/locations/{id}`	soft-delete	`location_not_found`

Self-protection rules:

- An admin cannot demote their own role (`cannot_demote_self`).
- An admin cannot deactivate their own account (`cannot_deactivate_self`).

Cross-org accesses return **`404 user_not_found / location_not_found`** —

same shape as "doesn't exist" so the API doesn't leak existence of other

orgs' data.

3. Event schema discipline

`app.api.routes.EVENT_SCHEMAS` is the single source of truth for

workflow events. New or unlisted types are rejected.

event_type	required keys	Who writes it
`encounter_created`	`status`	Server, on `POST /encounters`
`status_changed`	`old_status` , `new_status`	Server, on successful status transition
`note_draft_requested`	`requested_by`	Client (admin / clinician)
`note_draft_completed`	`template`	Client
`note_reviewed`	`reviewer`	Client
`manual_note`	`note`	Client — generic operator-authored event

Validation lives in `_validate_event(event_type, event_data)` :

- Unknown `event_type` → 400 `invalid_event_type` with the full

allowed list in the reason.

- `event_data` missing when required → 400 `invalid_event_data` .
- `event_data` not a JSON object → 400 `invalid_event_data` .
- Missing required keys → 400 `invalid_event_data` listing them.

Server-written events bypass the validator (they construct known-good

payloads directly). That keeps `encounter_created` / `status_changed`

hot paths free of needless checks while client-submitted events run

through a strict gate.

4. Pagination

`GET /encounters` now accepts:

param	default	range
`limit`	50	1 .. 500
`offset`	0	≥ 0

The response is still a JSON array (backward compatible), but three

headers carry the page metadata:

Header	Meaning
`X-Total-Count`	Full filtered count (pre limit/offset)
`X-Limit`	Echo of the applied limit
`X-Offset`	Echo of the applied offset

Existing clients that ignore headers keep working (they just see the

first page). The frontend client has a dedicated `listEncountersPage`

helper that surfaces the totals.

5. Frontend — admin UI

- Header shows an **Admin** button for role `admin` only.
- Clicking it opens a modal with two tabs: **Users** and **Locations**.
- **Users** tab: create form (email / full name / role), table with

inline role select + deactivate/reactivate buttons. Self-row is

disabled so admins can't lock themselves out. Role change goes

through `PATCH /users/{id}` with the exact pydantic model; failures

surface the `{error_code, reason}` envelope in a banner.

- **Locations** tab: create form (name), table with inline name edit

(click to edit; Enter saves, Escape cancels) and Deactivate buttons.

- Encounter list now paginates 25 rows at a time with Prev/Next

controls + "N-M of T" status line.

6. Tests

Backend (`apps/api/tests/test_admin.py`) — 20 tests

- DB role CHECK rejects unknown role (raises `IntegrityError`).
- Admin can create / update / deactivate users and locations in own org.
- Non-admin (clinician / reviewer) receives 403 `role_admin_required` on every admin write.
- Admin cannot create a user with an unknown role (`invalid_role`).
- Admin cannot duplicate emails (`user_email_taken`).
- Pydantic email validation returns 422.
- Admin cannot demote / deactivate self.
- Cross-org admin mutations return 404 `user_not_found` / `location_not_found` .
- Soft-deleted users/locations disappear from default list; `?include_inactive=1` shows them.
- `POST /encounters/{id}/events` rejects unknown `event_type` (`invalid_event_type`) and invalid `event_data` (`invalid_event_data`), accepts valid `manual_note` .
- Pagination: headers returned, offset works, filters + pagination combine correctly.

Frontend (`apps/web/src/test/AdminPanel.test.tsx`) — 5 Vitest tests

- Lists users on open.
- Create-user submit flows through `createUser` and shows success banner.
- Create-user 409 conflict surfaces error banner verbatim.
- Self-row role select + deactivate button are disabled.
- Create-location flow on the Locations tab.

Plus an App-level test asserting admin role sees the Admin button; clinician / reviewer do not.

E2E (`apps/web/tests/e2e/workflow.spec.ts`) — 2 new scenarios

- Admin opens the panel, creates a user and a location end-to-end against the live stack; both appear in their respective tables.
- Non-admin (clinician) never sees the Admin button.

7. What this phase explicitly does NOT do

- No org-level CRUD — `PATCH /organizations/{id}` is deliberately absent. Renaming an org is a rare event; add it when it's real.
- No per-org SSO configuration.
- No bulk-import.

- No invitation / email workflow.
- **Phase 13 update — audit log UI shipped.** The `security_audit_events` table is now exposed at `GET /security-audit-events` and surfaced in the Admin panel → Audit log tab with filters + pagination. See `23-operator-control-plane.md` .
- **Phase 14 update — real invitation workflow shipped.** `POST /users/{id}/invite` (admin) + `POST /invites/accept` (unauth, token IS the credential), 7-day expiry, sha256-only storage, per-user one-shot raw token banner, minimal `/invite?invite=<token>` accept screen. Bulk user import, audit CSV export, and typed org settings also landed this phase. Full contract: `24-invitations-and-governance.md` .

User lifecycle signal — `invited_at` (phase 13)

`users.invited_at` is stamped whenever an admin creates a user. The admin panel renders an "Invited" badge for active users with `invited_at` set. ChartNav does not send email; the badge is a real signal that the operator should communicate out-of-band and is not a token-based invitation flow. See `23-operator-control-plane.md` for the scope trade-offs.

Operator Control Plane

Phase 13 ships the pieces an operator actually needs to run the product:

organization settings, security-audit visibility, and a minimal user

lifecycle signal (`invited_at`). The admin panel becomes a four-tab

console: Users, Locations, Organization, Audit log.

1. Organization settings

API

Method	Path	Auth	Body
GET	`/organization`	any authed role	—
PATCH	`/organization`	admin only	`{ name?, settings? }`

Contract

- **Read** is open to any authenticated caller in the org — so the UI

can show org context in places the admin may not have visited.

- **Write** is `admin` -only (`role_admin_required` 403 otherwise).
- `slug` is **immutable** — renaming a slug ripples into URLs, audit

search, and any downstream integration keying on it. Documented on

the read response; the UI shows it readonly.

- `name` is a free text field (1..255 chars).
- `settings` is a JSON object. Server rejects non-object (pydantic

422) and blobs > 16 KB of serialized JSON (400 `settings_too_large`).

- PATCH only touches the caller's org; there is no path to mutate

another org (`organization_id` isn't even a parameter).

Persistence: new migration `d4e5f6a7b8c9` adds `organizations.settings TEXT NULL` .

2. Security audit read API

Endpoint

Method	Path	Auth
GET	`/security-audit-events`	admin only

Query parameters:

Param	Type	Notes
`event_type`	string	Exact match (e.g. `cross_org_access_forbidden`).
`error_code`	string	Exact match on the response `error_code` .
`actor_email`	string	Exact match.
`q`	string	Substring match against `path` or `detail` .
`limit`	int	1..500, default 50.
`offset`	int	≥0.

Response is a JSON array ordered newest-first (by `id DESC`). The pagination triplet (`X-Total-Count`, `X-Limit`, `X-Offset`) is returned as response headers, matching the encounter pagination contract.

Scoping rule (documented here so future readers don't have to guess)

```
WHERE organization_id = :caller.org OR organization_id IS NULL
```

Rows without an `organization_id` are pre-auth failures (no caller has been resolved yet — `missing_auth_header`, `unknown_user`, `invalid_token`, etc.). Those have **no** org attribution by construction, so showing them to every admin is correct: they're system-wide signals about unauth traffic hitting the service.

Rows **with** a cross-org attribution are never shown — if an admin in org1 issues a cross-org denial (actor=1, `organization_id=1`), it appears in org1's audit. If an attacker uses an org2 identity to probe org1, the audit row will be `organization_id=2` and admin1 will **not** see it. That's the right property: an org can't learn about sibling orgs by reading its own audit.

Backed by a pytest assertion (`test_audit_org_scoping`).

What gets written

No change from phase 10 — the HTTP exception handler still drives inserts via `app.audit.record(...)`. The read API simply surfaces what was already being persisted.

3. User lifecycle — `invited_at`

Migration `d4e5f6a7b8c9` adds `users.invited_at DATETIME NULL`. Admin create sets it to "now"; existing seeded users stay `null`. The admin panel renders an "Invited" badge on any active user with `invited_at` set.

Email delivery is **intentionally out of scope** — ChartNav doesn't send mail. The badge + timestamp are a real, testable signal that a user hasn't been "turned on" by any other channel yet; what the operator does with that (Slack, in-person, external mailer) is their business.

Out of scope:

- Token-based invitation links.
- Expiring invitations.
- Email delivery and templating.
- Self-serve signup.

4. Admin UI — four-tab console

`apps/web/src/AdminPanel.tsx` now has:

- **Users** — create form, table with inline role change, deactivate/reactivate, self-row disabled, "Invited" badge where `invited_at` is set.
- **Locations** — create form, inline rename, deactivate.
- **Organization** — loads `/organization`, form for `name` + `settings` JSON (16 KB cap enforced server-side); slug shown readonly; local JSON parse error surfaces inline before any PATCH is fired.
- **Audit log** — filter row (`event_type`, `actor_email`, free-text `q`), paginated table (25/page), timestamp / event / actor / method+path / error code / request id columns. Backend 403 surfaces as banner.

UX invariants preserved from earlier phases:

- Non-admin callers never see the Admin button, and the panel never renders for them.
- All mutations disable submit while in flight.
- Every 4xx surfaces the backend `{error_code, reason}` envelope.

5. Tests

Backend (`apps/api/tests/test_control_plane.py`, 17 new)

- Read `/organization` for all three roles + 401 without auth.
- Admin patches name, settings (valid JSON object).
- Non-object settings → 422.
- Settings > 16 KB → 400 `settings_too_large`.
- Non-admin PATCH → 403 `role_admin_required`.
- Cross-org isolation: admin1 PATCHing org1 does not change org2's row.
- Admin reads `/security-audit-events`.
- Non-admin 403.
- Filters by `event_type`, `actor_email`, and `q` substring match.
- Pagination (limit/offset, disjoint pages, X-Limit/Offset headers).
- Org scoping: admin1 never sees rows with `organization_id` outside `{1, null}`.
- New users carry `invited_at`; seeded ones do not.

Frontend (`apps/web/src/test/AdminPanel.test.tsx`, 4 new on top of phase 12)

- Organization tab loads current org, readonly slug, edits dispatch PATCH.
- Organization settings local JSON parse error surfaces before calling backend.
- Audit tab loads, row rendered with `event_type`, filter dispatch hits the mocked API.
- Audit 403 surfaces as error banner.

E2E (`apps/web/tests/e2e/workflow.spec.ts`, 1 new)

- Admin opens panel → Organization tab → edits name → Audit tab loads. End-to-end against the live backend.

Totals: **pytest 88/88, Vitest 22/22, Playwright 11/11**.

6. What this phase does NOT do

- No row-level user actions on the audit log (expand a row, pin, comment). The table is read-only.
- No export of audit events.
- No CSV download for users/locations/audit.
- No rich settings schema — the app doesn't consume any specific key in `settings` yet. The field exists so operators can stash tenant preferences before we build the consumer.

Invitations, Governance & Bulk Operations

Phase 14 closes the "scaffolding only" loops from the operator

control plane:

- invitations are a real token flow (admin issues, user accepts) —

no email delivery, but everything else operates like production;

- organization `settings` is typed and extra-forbidden (plus an

`extensions` bucket for forward-compat drift);

- audit events export to CSV;
- `event_data` payloads now enforce value types / enum membership;
- bulk user import lands (JSON body, per-row pass/fail).

1. Invitation workflow

Data shape (migration `e5f6a7b8c9d0`)

Three new columns on `users` :

- `invitation_token_hash TEXT NULL` — **sha256 hex only**; the raw

token is never stored. Indexed for O(1) accept lookups.

- `invitation_expires_at DATETIME NULL` — 7 days from issue.
- `invitation_accepted_at DATETIME NULL` — set on successful accept;

presence blocks re-issue.

API

Method	Path	Auth	Notes
POST	`/users/{id}/invite`	admin	Issues (or re-issues) a token. Returns raw token ONCE. Inactive user → 400 `user_inactive` . Already-accepted → 400 `user_already_accepted` . Cross-org → 404 `user_not_found` .
POST	`/invites/accept`	unauth	Body: `{ token }` . Invalid / expired / replayed → 400 `invalid_invite` / `invite_expired` . On success the hash is cleared so the same token can't be reused.

Security notes

- Only a sha256 hash is persisted. The raw token is echoed back in

the admin response and is expected to be shared out-of-band.

- Tokens are generated via `secrets.token_urlsafe(32)` (≥ 256 bits

of entropy).

- `/invites/accept` is rate-limited via the middleware's protected

prefix list (`/invites`).

- Re-issuing an invite overwrites the hash: the old token is

immediately invalid.

- Accept is idempotent in the wrong direction — a second accept

with the same token fails with `invalid_invite` because the hash

was cleared on the first success.

UI

- Admin panel → Users tab: every active, non-self user row gets an

Invite button. Clicking it surfaces the raw token in a

one-shot banner with a "Dismiss" button; the user is told to

copy immediately.

- Minimal `/invite?invite=<token>` accept screen in the frontend

(no router dependency; see `apps/web/src/InviteAccept.tsx` +

`main.tsx` branch). On success it stores the accepted email as

`localStorage.chartnav.devIdentity` so the main app picks it up

for header-mode dev use.

Intentionally out of scope

- Email delivery / templating.
- Self-serve signup.
- IdP pre-provisioning (the app will always map back through the

`users` table; that's the production-auth claim seam).

2. Organization settings — typed schema

`app.api.routes.OrganizationSettings` (pydantic `extra=forbid`):

field	type	notes
`default_provider_name`	`str` ≤ 255	suggestion used by UI for create flows
`encounter_page_size`	`int 10..200`	caller-facing default page size
`audit_page_size`	`int 10..200`	audit-log default page size
`feature_flags`	`dict[str, bool]`	on/off toggles
`extensions`	`dict[str, Any]`	explicit forward-compat bucket

PATCH `/organization` body:

```
{
  "name": "Demo Eye Clinic",
  "settings": {
    "default_provider_name": "Dr. Carter",
    "encounter_page_size": 50,
    "feature_flags": {"beta_ui": true},
    "extensions": {"brand_color": "#0B6E79"}
  }
}
```

Validation:

- Top-level unknown keys → 422 (pydantic `extra=forbid`).
- Oversized blob (≥ 16 KB after JSON serialization) → 400 `settings_too_large` .
- Stored as JSON text in `organizations.settings` ; normalized via

`model_dump(exclude_none=True)` so the blob only contains keys the

operator actually set.

UI (`AdminPanel.tsx` → Organization tab): dedicated inputs for each

typed field, plus an **Extensions** textarea for the forward-compat

bucket. The frontend no longer exposes a raw settings textarea.

3. Audit export

`GET /security-audit-events/export` — admin only. Honors the same

filters as the audit read endpoint (`event_type` , `error_code` ,

`actor_email` , `q`) and the same org-scoping rule (caller's org OR

`NULL`). Returns CSV with a stable column order:

```
id, created_at, event_type, error_code, actor_email, actor_user_id,
organization_id, method, path, request_id, remote_addr, detail
```

Browser download: Content-Disposition attachment with a timestamped

filename (`chartnav-audit-YYYYMMDDTHHMMSSZ.csv`).

Frontend: admin panel → Audit log tab has an **Export CSV** button

that fetches with the auth header and triggers a local download

(required because `<a href>` alone can't send `X-User-Email`).

4. Event payload hardening

Beyond "required keys exist" (phase 12), each event type now enforces

value types / enum membership:

event_type	hardening
`status_changed`	`old_status` & `new_status` must be in `ALLOWED_STATUSES`
`encounter_created`	`status` must be in `ALLOWED_STATUSES`
`manual_note`	`note` must be a non-empty string ≤ 4000 chars
`note_draft_requested`	`requested_by` non-empty string; optional `template` non-empty string
`note_draft_completed`	`template` non-empty string; optional `length_words` non-negative int
`note_reviewed`	`reviewer` non-empty string ≤ 255

Any violation → 400 `invalid_event_data` with a specific reason.

5. Bulk user import

`POST /users/bulk` (admin only) accepts up to 500 rows at a time.

Request body:

```
{
  "users": [
    { "email": "a@x.test", "full_name": "A", "role": "clinician" },
    { "email": "b@x.test", "role": "reviewer" }
  ]
}
```

Response:

```
{
  "created": [/* full user rows */],
  "skipped": [{ "row": 0, "email": "admin@chartnav.local", "error_code": "user_email_taken" }],
  "errors":  [{ "row": 1, "email": "x@y.test", "error_code": "invalid_role" }],
  "summary": { "requested": 4, "created": 2, "skipped": 1, "errors": 1 }
}
```

Rules:

- Per-row validation — one bad row does not abort the batch.
- Strictly org-scoped: every created row belongs to `caller.organization_id` .

- `invited_at` stamped on every newly created row (bulk import is by

definition an "invitation needed" event).

- Admin-only (`role_admin_required` 403 otherwise).
- Empty batch → 422 (pydantic `min_length=1`).

UI: Users tab → **Bulk import…** opens a dialog with a CSV-like

textarea (`email,full_name,role` per line; header row optional). On

submit the dialog surfaces created / skipped / errors inline and then

refreshes the list.

6. Tests

Backend (`apps/api/tests/test_invitations.py` , 20 new)

Invite issue + accept happy path, invalid/expired/replayed/reissued

tokens, cross-org denial, inactive-user denial, already-accepted

denial. Audit CSV: admin-only, honors filters, has expected shape.

Event hardening: bogus status values, empty/non-string `note` ,

empty `requested_by` . Bulk import: created/skipped/errors summary,

org-scoped, admin-only, empty-body 422.

Frontend (`apps/web/src/test/AdminPanel.test.tsx` , 3 new on top of phase 13)

- `admin-user-invite-<id>` surfaces the one-time raw token banner.
- Bulk import summary renders created/skipped/errors counts.
- Audit export button wires `downloadAuditExport` with current filters.

E2E (`apps/web/tests/e2e/workflow.spec.ts` , 1 new)

- Admin creates a new user, clicks **Invite**, raw token appears.

Audit log → **Export CSV** triggers a real browser download with

the expected filename pattern.

Totals: **pytest 110/110, Vitest 25/25, Playwright 12/12**.

7. What this phase explicitly does NOT do

- No email delivery for invitations.
- No SSO claim → user mapping changes (the auth seam from phase 10 still applies).
- No organization or audit data export beyond users (CSV export is audit-only).
- No role-based access for the invite accept URL (anyone with the token can accept; that's how invitations work by design).
- No CRUD for the `security_audit_events` table from the API — read + export only.

Enterprise Quality & Compliance

Phase 15 turns ChartNav from "enterprise-shaped" into "enterprise-credible" by landing five real guarantees: an a11y floor, a visual-regression baseline, a release SBOM + image digest, an audit retention helper, and admin-list pagination/search — plus two feature flags that actually gate UI behavior.

1. Accessibility baseline

Stack: `@axe-core/playwright` over Chromium.

Scope: app shell + encounter list + encounter detail + admin panel (users + audit tabs) + invite accept screen.

Tests: `apps/web/tests/e2e/a11y.spec.ts` (5 scenarios). The floor is the set of `serious` and `critical` axe findings — minor ones are logged but not blocking. Local + CI.

Fixes landed while establishing the baseline:

- `aria-label="Event type"` on the event-composer `<select>` in encounter detail.
- `aria-label="Role for <email>"` on each inline role `<select>` in the admin users table.

Commands:

```
make e2e-a11y          # only axe scans
make e2e               # full suite including a11y
```

Gaps intentionally left:

- No color-contrast audit beyond axe's automated rules (manual pass not done).
- No keyboard-only walkthrough of every modal (would warrant a dedicated QA sweep).

2. Visual regression baseline

Stack: Playwright `toHaveScreenshot`.

Scope: `apps/web/tests/e2e/visual.spec.ts` — 4 anchor views (encounter list, admin users tab, admin audit tab, invite accept).

Setup notes:

- Viewport pinned to 1280×820.
- Animations + transitions disabled via an injected stylesheet.
- Identity seeded to `admin@chartnav.local` before each snapshot.
- `maxDiffPixelRatio: 0.02` — small renderer drift shouldn't break a run, but a real layout regression will cross it.

Baselines are **OS-specific** (Chromium renders subtly differently on macOS vs. Linux). We ship macOS baselines under `apps/web/tests/e2e/visual.spec.ts-snapshots/*-chromium-darwin.png` for local dev. CI does **not** run visual regression (it would fail first-run against mismatched-OS baselines); it is a local-only gate for now. Documented honestly as a remaining gap.

Commands:

```
make e2e-visual          # run against current baselines
make e2e-visual-update   # intentional UI change → refresh baselines
```

3. Admin list scaling

Backend: `GET /users` and `GET /locations` now accept `limit` (1..500, default 100), `offset` (≥ 0), `q` (substring search), plus `role` on `/users`. They emit `X-Total-Count`, `X-Limit`, `X-Offset` headers. All filters combine with the existing `include_inactive` lens.

Frontend: the Users and Locations tabs have a search box + Prev/Next pager (25/page) + count header. Org scope is preserved — pagination never leaks cross-org rows.

Tests: `test_enterprise.py` covers pagination headers, offset movement, `q` substring match, role filter (valid + invalid), cross-org isolation.

4. Feature-flag consumption

`organization.settings.feature_flags` is now consumed by the frontend. Two flags ship wired:

flag	behavior when `false`
`audit_export`	Hides the **Export CSV** button on the Audit tab
`bulk_import`	Hides the **Bulk import...** button on Users tab

Both default to `true` (absent → on). Admins flip them from the Organization tab. Server is unchanged — the endpoints still accept requests from admins regardless of flag state. Flags only affect the UI surface; a sophisticated attacker with an admin role could still hit the endpoint directly. That's the point of feature flags as UX toggles, not security controls, and it's documented that way.

Frontend test coverage (`AdminPanel.test.tsx`):

- `audit_export=false` → `admin-audit-export` absent, `admin-audit-refresh` present.
- `bulk_import=false` → `admin-user-bulk-open` absent.
- Search field dispatches `listUsersPage({ q: ... })` .

5. Release compliance signals

SBOM

`scripts/sbom.py` writes a JSON document with:

- project + version + git sha/ref/tag/dirty-flag,
- image owner + tag (when `CHARTNAV_IMAGE_TAG` is set),
- full `pip list --format json` of the API venv,
- full `npm list --all --json` tree of `apps/web` (falls back to

`package-lock.json` summary if node_modules isn't installed).

Explicit note on the file itself: this is **not** a signed CycloneDX

document. It's a real inventory, honest about its shape. Sufficient

for audit questions about dep versions + git provenance; upgrading to

signed CycloneDX (`cyclonedx-py` + `cyclonedx-npm`) is the obvious

next step and the `.notes` field in the SBOM says so.

Image digest capture

`scripts/release_build.sh` writes `chartnav-api-<version>.digest.txt`

via `docker image inspect --format '{{.Id}}'` . The digest is attached

to GitHub Releases alongside the tarballs so deployers can verify the

pulled image id against what was released.

Release bundle contents (phase 15)

`dist/release/<version>/` :

- `chartnav-api-<version>.tar` (docker save)
- `chartnav-web-<version>.tar.gz` (vite build)
- `chartnav-staging-<version>.tar.gz` (compose + scripts + docs)
- `chartnav-sbom-<version>.json` **NEW**
- `chartnav-api-<version>.digest.txt` **NEW**
- `MANIFEST.txt` (sha256 of every file)

`MANIFEST.txt` now sha256s every artifact including the SBOM + digest

file, so the manifest itself is the integrity anchor.

6. Retention / compliance scaffolding

Audit retention helper

`apps/api/app/retention.py::prune_audit_events(retention_days, dry_run)`

deletes rows from `security_audit_events` older than `retention_days` .

The app itself **never** silently prunes — the helper runs only when

an operator invokes it. Retention is driven by

`CHARTNAV_AUDIT_RETENTION_DAYS` (default `0` = never prune).

Operator CLI

```
# use env default
python scripts/audit_retention.py
# explicit threshold
python scripts/audit_retention.py --days 90
# report without deleting
python scripts/audit_retention.py --days 90 --dry-run
```

```
# via Make
make audit-prune ARGS="--days 90 --dry-run"
```

Output is a JSON summary: `status`, `retention_days`, `cutoff`,

`matched`, `deleted`, `dry_run`. Backend tests (`test_enterprise.py`)

cover disabled (`0`), dry-run, and real deletion paths.

Runbook notes

See `21-staging-runbook.md` for the cron-friendly invocation and the

compliance-facing retention discussion in `20-observability.md`.

What this scaffold does NOT do

- No SIEM shipping / archival-to-S3 out of the box.
- No regulated-industry attestation (HIPAA/SOC2/ISO) — documentation

only. Those belong to a real audit workstream.

- No row-level "legal hold" mechanism.

7. Test + CI summary

Layer	Count	Notes
pytest (backend)	118	+8 `test_enterprise.py` (admin list + retention + feature-flag round-trip)
Vitest (frontend)	28	+3 AdminPanel (flags, search dispatch)
Playwright E2E	21	+5 a11y (axe) + 4 visual regression + 12 workflow

CI (`.github/workflows/ci.yml`):

- The existing `e2e` job now runs `workflow.spec.ts` **and**

`a11y.spec.ts` (hard gate).

- Visual regression is local-only (see above).
- Release workflow (`release.yml`) bundles SBOM + image digest;

operators receive them in every GitHub Release.

8. What this phase explicitly does NOT do

- No Sigstore/cosign signing of images or artifacts.
- No CycloneDX / SPDX standardized SBOM.
- No attested provenance (SLSA, in-toto).
- No automated dependency-vulnerability scan (depends on the

organization's chosen scanner — the SBOM is the input).

- Visual regression does not run in CI.
- Retention helper is not scheduled in the app itself — operator cron

is expected.

26 — Platform mode & interoperability

Platform Mode & Interoperability

ChartNav ships with an explicit architectural seam so it can run as a standalone lightweight EMR/EHR **or** sit on top of an existing EMR/EHR as a workflow / documentation / orchestration layer. No code path assumes one deployment shape.

1. Operating modes

Driven by `CHARTNAV_PLATFORM_MODE`. Three values, each with a concrete contract:

Mode	ChartNav is ...	Clinical SoR	Writes to external system?
`standalone`	the system of record	ChartNav DB	N/A
`integrated_readthrough`	a read-through layer + native workflow	external EHR/EMR	**No** (explicitly refused)
`integrated_writethrough`	a read + write layer	external EHR/EMR (mirrored in ChartNav when cached)	**Yes** (via the adapter)

Selection:

```
# Standalone (default — current build)
export CHARTNAV_PLATFORM_MODE=standalone

# Read-only overlay on top of an existing EHR/EMR
export CHARTNAV_PLATFORM_MODE=integrated_readthrough
export CHARTNAV_INTEGRATION_ADAPTER=stub   # or a vendor key

# Read + write overlay
export CHARTNAV_PLATFORM_MODE=integrated_writethrough
export CHARTNAV_INTEGRATION_ADAPTER=stub   # or a vendor key
```

The config layer (`app/config.py`) validates the combination at import time and fails loudly on mismatch (e.g. `standalone + adapter=stub` → RuntimeError).

2. Adapter layer

The boundary between ChartNav's core and an external clinical system is a single Python protocol: `app.integrations.base.ClinicalSystemAdapter`.

Core services **never** talk to a vendor SDK, DB driver, or HTTP client directly. They talk to an adapter.

2.1 Shipped adapters

Key	Module	Use	`supports_encounter_write`
`native`	`app/integrations/native.py`	Standalone — ChartNav DB is SoR	✓
`stub`	`app/integrations/stub.py`	Integrated without a real vendor yet	✓ (write-through only)

Both implementations are honest:

- `NativeChartNavAdapter` reuses the same SQLAlchemy Core query surface the HTTP routes use. It refuses `fetch_patient` / `search_patients` today because a native `patients` table is not yet modeled — documented and test-asserted.
- `StubClinicalSystemAdapter` exposes a canned read shape so the UI can render an integrated deployment end-to-end. In read-through mode it raises `AdapterNotSupported` on every write; in write-through mode it records writes to an in-process list (inspectable in tests) but never pretends to have reached a real external system.

2.2 Adding a vendor adapter

1. Create `app/integrations/<vendor>.py`. Implement `ClinicalSystemAdapter`. Raise `AdapterError("error_code", "…")` for transport/auth/validation failures and `AdapterNotSupported` for operations the vendor doesn't support.
2. Register a factory:

```python
from app.integrations import register_vendor_adapter
from app.integrations.vendor_x import VendorXAdapter
register_vendor_adapter("vendor_x", lambda: VendorXAdapter())
```

3. Set `CHARTNAV_INTEGRATION_ADAPTER=vendor_x`. Nothing else in the core changes.

The registry is a plain dict — deliberately mutable, deliberately grep-able. No decorator magic, no import-time side effects hidden in the core.

2.3 What the contract expresses

The protocol is small by design. Current surface:

- `fetch_patient(patient_id)` / `search_patients(query, limit)`
- `fetch_encounter(encounter_id)`
- `update_encounter_status(encounter_id, new_status, *, changed_by)`
- `write_note(encounter_id, author_email, body, note_type)`
- `sync_reference_data()`
- `info` → `AdapterInfo` (supports-* flags + source-of-truth map)

Adding operations is cheap; removing them later is hard. Resist speculative additions.

3. Source-of-truth model

Each shipped adapter declares, per domain object, who owns the canonical copy:

Object	`native`	`stub` (integrated)
organization	ChartNav	mirrored
location	ChartNav	mirrored
user	ChartNav	ChartNav
encounter	ChartNav	external
workflow_event	ChartNav	ChartNav
patient	not supported	external
document	ChartNav	external

This map is surfaced at `GET /platform` and rendered in the admin

panel so the semantics of a given install are never implicit.

4. Request flow

```
flowchart LR UI[Admin / clinician UI] -->|HTTP| API API -->|service layer| CORE[Core domain services] CORE -->|ClinicalSystemAdapter| ADAPTER ADAPTER --> NATIVE[(Native ChartNav DB)] ADAPTER --> VENDOR[(External EHR/EMR)] ADAPTER --> STUB[(Stub / in-memory)]
```

In standalone mode the adapter path terminates at the native DB;

in integrated modes it terminates at the vendor (or the stub).

5. Runtime validation

At import time `app/config.py` enforces:

- `CHARTNAV_PLATFORM_MODE` ∈ {standalone, integrated_readthrough, integrated_writethrough}.
- `standalone` **forbids** any adapter other than `native` .
- `integrated_*` defaults the adapter to `stub` when unset.

At adapter resolution time `app/integrations/__init__.py::resolve_adapter` :

- Returns the singleton matching the configuration.
- Raises `RuntimeError` with an actionable message when an

integrated mode asks for a vendor key that isn't registered.

6. HTTP surface

`GET /platform` (auth required):

```
{
  "platform_mode": "standalone",
  "integration_adapter": "native",
  "adapter": {
    "key": "native",
    "display_name": "ChartNav native",
    "description": "…",
    "supports": {
      "patient_read": false,
      "patient_write": false,
      "encounter_read": true,
      "encounter_write": true,
      "document_write": true
    },
    "source_of_truth": {
      "encounter": "chartnav",
      "patient": "not_supported",
      "…": "…"
    }
  }
}
```

No secrets leak — JWT config, DB URL, and vendor credentials never appear on this surface. The test suite asserts that explicitly.

7. Frontend awareness

The admin panel renders a **platform banner** at the top (inside the modal, above the tabs) on every admin view:

```
Platform mode: Standalone (ChartNav-native) · ChartNav native
```

Derived entirely from `GET /platform`. Changes the minute an operator flips `CHARTNAV_PLATFORM_MODE` and restarts the API. Frontend test coverage in `src/test/AdminPanel.test.tsx` proves standalone and integrated-readthrough render distinctly.

8. What this phase explicitly does NOT do

- Does **not** implement a real Epic / Cerner / Nextech / Athena adapter. The contract exists; vendor work plugs into the registry.
- Does **not** ship a native `patients` table. Standalone mode owns encounters, workflow_events, users, locations, organizations, and documents-as-workflow-events today. Patient modeling is the obvious next standalone-mode build.
- Does **not** implement FHIR. The adapter contract maps cleanly to a subset of FHIR resources (Patient, Encounter, DocumentReference), but translating the wire format is the vendor adapter's job.
- Does **not** implement conflict resolution between mirrored caches and external writes. That's a real integration concern that belongs to each vendor adapter, not the core.

9. Adoption path

See `27-adoption-and-implementation-model.md` for how a clinic moves through these modes over time.

27 — Adoption & implementation model

Adoption & Implementation Model

How a clinic moves onto ChartNav without ripping out what they have.

1. Adoption paths

ChartNav supports **two entry points** — neither is a "rip and replace":

Path A — Overlay (integrated mode)

The clinic keeps its existing EHR/EMR. ChartNav runs next to it and reads through an adapter for encounters/patients, while writing workflow, coding, and documentation activity back into ChartNav's own tables. Over time, more operations can be pushed back through the adapter into the incumbent system.

```
Day 1   → CHARTNAV_PLATFORM_MODE=integrated_readthrough  (observe-only)
Day 30  → CHARTNAV_PLATFORM_MODE=integrated_writethrough (push notes / status)
Day 90+ → add coding push, reference data sync, etc. via vendor adapter
```

Path B — Standalone (adopt-as-EMR mode)

The clinic has no incumbent EHR/EMR, or is replacing one. ChartNav is the system of record. The native adapter persists everything to the ChartNav database.

```
Day 1   → CHARTNAV_PLATFORM_MODE=standalone
Day 30  → native patients table (next build phase)
Day 90+ → full documentation, coding, billing extensions
```

Both paths use **the same ChartNav core**. The difference is adapter configuration, not code.

2. What ChartNav owns vs what the EMR owns

Object	`standalone`	`integrated_readthrough`	`integrated_writethrough`
Organization / tenant	ChartNav	ChartNav (mirrored)	ChartNav (mirrored)
Location	ChartNav	ChartNav (mirrored)	ChartNav (mirrored)
User / role / auth	ChartNav	ChartNav	ChartNav
Patient	*(roadmap)*	External EHR	External EHR
Encounter	ChartNav	External EHR (fetched via adapter)	External EHR (updated via adapter)
Workflow events	ChartNav	ChartNav	ChartNav
Coding / billing metadata	ChartNav	ChartNav (surface)	Pushed through adapter
Documents / notes	ChartNav (as workflow_events)	External EHR (read-only)	Pushed through adapter
Audit trail	ChartNav	ChartNav	ChartNav

Audit and identity are **always ChartNav-owned**. That boundary is intentional: every operator action through the ChartNav surface

needs to be auditable regardless of whether the underlying write landed in the EHR or the ChartNav DB.

3. Implementation sequence

Recommended sequence for a vendor adapter:

1. **Read-only scan.** Implement `fetch_patient`, `fetch_encounter`, `search_patients`. Boot in `integrated_readthrough`. Operator can verify patient/encounter lookups without a risk of writing.
2. **Reference sync.** Implement `sync_reference_data`. Populate ChartNav-side provider and location caches so the UI isn't blank.
3. **Status write.** Implement `update_encounter_status`. Flip to `integrated_writethrough`. The operator now drives the incumbent EMR from ChartNav's workflow UI.
4. **Documentation push.** Implement `write_note`. ChartNav becomes the documentation surface; notes land in the incumbent EMR.
5. **Coding / billing.** Vendor-specific and out of scope for the core protocol today — extend `ClinicalSystemAdapter` with a targeted method when a concrete caller exists. Don't speculate.

Each step is independently shippable. A clinic can stop at any point.

4. Deployment posture per mode

Concern	`standalone`	`integrated_*`
DB	Postgres (ChartNav)	Postgres (ChartNav) + vendor transport
Auth	ChartNav JWT / header	ChartNav JWT / header
Backup / DR	Clinic owns the DB	Clinic owns ChartNav DB; vendor owns EMR data
PHI surface	All clinical data in ChartNav	Clinical data transits via adapter; cached in ChartNav only when necessary
Regulatory posture	Full EMR / EHR	Workflow overlay (regulatory surface follows the incumbent)

5. Migration out

Because standalone data is persisted in a plain Postgres schema described in `04-data-model.md`, an eventual export to a commercial EMR is a SQL job + mapping table, not a proprietary migration. Forward compatibility is explicit: forward-only Alembic migrations, clear column semantics, no opaque blobs.

6. What ChartNav is **not** claiming

- Not a certified EHR today.
- Not HIPAA/SOC2/ISO attested by a third party today — those belong to the deployment context, and the platform's operational hardening (audit, RBAC, JWT, request tracing) is the input, not

the certification.

- Not a turnkey FHIR server — the adapter boundary is FHIR-friendly,

but the translation is the vendor adapter's job.

This document is the operator-facing contract. The engineering

contract lives in `26-platform-mode-and-interoperability.md`.

Diagram — System architecture

System Architecture

```
flowchart LR subgraph Web["Frontend (apps/web, Vite + React + TS)"] UI["App.tsx
list · detail · timeline · actions"] ApiClient["api.ts
typed client + ApiError"] Ident["identity.ts
localStorage dev caller"] end subgraph Runtime["Backend (apps/api, FastAPI)"] FastAPI["main.py
+ exception handler → audit"] RID["RequestIdMiddleware"] Access["AccessLogMiddleware
(JSON logs)"] Rate["RateLimitMiddleware
(60s window)"] Audit[("security_audit_events")] Metrics["metrics.py
(chartnav_* counters)"] Ready["/ready (DB ping)"] Config["config.py
env-driven settings"] Authn["require_caller
header · bearer (PyJWKClient)"] Authz["authz.py RBAC"] Router["routes.py"] SM["State Machine"] DB["db.py · SA Core"] Engine[(SA Engine)]
Adapters["app/integrations
ClinicalSystemAdapter"] Native["NativeChartNavAdapter"] Stub["StubClinicalSystemAdapter"] Vendor[("VendorAdapter
(registry; empty today)")] end subgraph Storage SQLite[("SQLite
apps/api/chartnav.db")] PG[("Postgres 16")] end subgraph Deploy Dockerfile["Dockerfile
non-root · healthcheck"] Entry["entrypoint.sh
migrate + exec"] Compose["docker-compose.prod.yml"] StageCompose["docker-compose.staging.yml
pinned tag · /ready healthcheck"] StageScripts["staging_up · staging_verify
staging_rollback"] end subgraph Tooling Alembic["Alembic
DATABASE_URL-aware"] Seed["scripts_seed.py"] PyTest["pytest (28)"] VitestSuite["vitest (12)"] Playwright["Playwright E2E (8)"]
Smoke["scripts/smoke.sh"] Verify["scripts/verify.sh"] PgVerify["scripts/pg_verify.sh"] DocBuild["scripts/build_docs.py"]
ReleaseScript["scripts/release_build.sh"] Make["Makefile
install · verify · pg-verify · dev · web-* · e2e · release-build"] CI[".github/workflows/
ci.yml · release.yml"] end UI --> ApiClient ApiClient -->|"HTTP + X-User-Email or Bearer"| RID RID --> Access Access --> Rate Rate --> FastAPI
FastAPI -. audits denials .-> Audit FastAPI -. bumps .-> Metrics Access -. observes .-> Metrics Rate -. observes .-> Metrics FastAPI --> Ready
Metrics -->|"/metrics text"| ApiClient Ident -. stored caller .-> ApiClient FastAPI --> Authn Authn --> Authz Authz --> Router Router --> SM
Router --> DB Router --> Adapters Adapters --> Native Adapters --> Stub Adapters --> Vendor Native --> DB DB --> Engine Engine --> SQLite
Engine --> PG Config -.-> Authn Config -.-> DB Config -.-> Adapters Alembic -.-> Engine Seed -.-> DB PyTest -.-> FastAPI VitestSuite -.-> UI
Playwright -.->|"live browser"| UI Playwright -.->|"live HTTP"| FastAPI Smoke -.-> FastAPI Verify --> Make PgVerify --> Make DocBuild -->
Make Make --> Alembic Make --> Seed Make --> PyTest Make -. web-dev · web-build .-> UI CI --> Make Dockerfile --> Entry Entry -->
Alembic Entry --> FastAPI Compose --> Dockerfile Compose --> PG
```

Diagram — Encounter status machine

Encounter Status State Machine

```
stateDiagram-v2 [*] --> scheduled [*] --> in_progress : walk-in scheduled --> in_progress in_progress --> draft_ready draft_ready --> review_needed draft_ready --> in_progress : rework review_needed --> completed review_needed --> draft_ready : kick back completed --> [*]
```

Legend: any edge not shown is rejected at `POST /encounters/{id}/status`

with HTTP 400 `invalid_transition` .

ER Diagram

erDiagram organizations ||--o{ locations : "has" organizations ||--o{ users : "employs (identity + role)" organizations ||--o{ encounters : "owns" locations ||--o{ encounters : "hosts" encounters ||--o{ workflow_events : "emits" security_audit_events { int id PK string event_type string request_id string actor_email int actor_user_id int organization_id string path string method string error_code text detail string remote_addr datetime created_at } organizations { int id PK string name string slug UK datetime created_at } locations { int id PK int organization_id FK string name boolean is_active datetime created_at } users { int id PK int organization_id FK string email UK string full_name string role "admin | clinician | reviewer (CHECK)" boolean is_active datetime invited_at string invitation_token_hash "sha256 hex, indexed" datetime invitation_expires_at datetime invitation_accepted_at datetime created_at } encounters { int id PK int organization_id FK int location_id FK string patient_identifier string patient_name string provider_name string status datetime scheduled_at datetime started_at datetime completed_at datetime created_at } workflow_events { int id PK int encounter_id FK string event_type text event_data datetime created_at }

Seeded tenants & users

org_id	slug	email	role
1	`demo-eye-clinic`	admin@chartnav.local	admin
1	`demo-eye-clinic`	clin@chartnav.local	clinician
1	`demo-eye-clinic`	rev@chartnav.local	reviewer
2	`northside-retina`	admin@northside.local	admin
2	`northside-retina`	clin@northside.local	clinician

`users.email` is the authentication key consumed from `X-User-Email`.

`users.role` is the RBAC key consumed by `app.authz`.

`users.organization_id` is the authoritative source of scope; never

derived from client input.

Diagram — API / data flow

API / Data Flow & CI

UI → Backend request path

sequenceDiagram autonumber participant U as Operator (browser) participant W as apps/web (App.tsx + api.ts) participant Cfg as app.config participant Authn as require_caller participant Authz as authz gate participant R as Route handler participant SM as State Machine participant DB as SA Core → SQLite / Postgres U->>W: pick identity · change filter · click transition W->>Authn: fetch with X-User-Email: admin@chartnav.local Cfg-->>Authn: auth_mode=header Authn->>DB: SELECT user WHERE email=:email DB-->>Authn: {role, org} Authn-->>Authz: Caller Authz->>R: Caller R->>DB: scoped SQL (:name binds) DB-->>R: rows R-->>W: 200 + JSON (or 4xx {error_code, reason}) W-->>U: render list · detail · timeline · banner Note over W,R: On 4xx, api.ts wraps into ApiError(status, error_code, reason). App.tsx surfaces it verbatim as a banner.

Role-aware transition flow

sequenceDiagram autonumber participant W as App.tsx participant A as api.allowedNextStatuses() participant B as POST /encounters/{id}/status participant Authz as authz.assert_can_transition W->>A: compute buttons for (role, current) A-->>W: [next1, next2, ...] (UI hint only) W->>B: POST status=next B->>Authz: validate state-machine edge + role edge alt allowed Authz-->>B: ok B-->>W: 200 updated encounter W->>W: refresh detail + events + list; green banner else role forbidden Authz-->>W: 403 role_cannot_transition W->>W: red banner; UI affordances unchanged until next refresh end

CI gate flow

flowchart TD PR[["push / PR"]] --> SQL[backend-sqlite] SQL --> PG[backend-postgres] SQL --> DK[docker-build] SQL --> DOC[docs] subgraph "backend-sqlite" A1[pip install -e .dev,postgres] --> A2[alembic upgrade] A2 --> A3[seed x2] A3 --> A4[pytest 28] A4 --> A5[scripts/verify.sh] end subgraph "backend-postgres (service: postgres:16-alpine)" B1[install] --> B2[alembic upgrade] B2 --> B3[seed x2] B3 --> B4[boot + smoke] B4 --> B5[live status transition] end subgraph "docker-build" D1[buildx build] --> D2[run container] D2 --> D3[smoke] end subgraph "docs" X1[apt chromium] --> X2[build_docs.py] X2 --> X3[upload artifact] end

Bearer mode + audit trail (phase 10)

sequenceDiagram autonumber participant C as Client participant RID as RequestIdMiddleware participant Rate as RateLimitMiddleware participant Authn as require_caller (bearer) participant JWKS as JWKS URL (PyJWKClient) participant R as Route participant EH as HTTPException handler participant Audit as security_audit_events C->>RID: GET /me
Authorization: Bearer RID->>Rate: request + request.state.request_id Rate->>Authn: request (within window) Authn->>JWKS: get_signing_key_from_jwt(token) JWKS-->>Authn: public key Authn->>Authn: jwt.decode — sig + iss + aud + exp alt invalid token / iss / aud / expired / missing claim / unknown user Authn-->>EH: HTTPException(status=401, error_code=...) EH->>Audit: record(event_type, request_id, path, method, remote_addr, ...) EH-->>C: 401 + X-Request-ID else valid Authn->>Authn: SELECT users WHERE email = claim Authn-->>R: Caller(user_id, role, organization_id) R-->>C: 200 + X-Request-ID end

Staging deploy flow (phase 11)

flowchart LR Op[Operator] --> Up[scripts/staging_up.sh] Up --> V[docker compose config] V --> UP[docker compose up -d] UP --> API[api container
pinned ghcr.io/.../chartnav-api:$TAG] UP --> DB[(postgres:16-alpine)] API --> E[entrypoint.sh
alembic upgrade + uvicorn] E --> Ready["HEALTHCHECK
GET /ready"] Ready -->|ok| Proxy[reverse proxy] Op --> Verify[scripts/staging_verify.sh] Verify -->|health · ready · metrics · 401 · x-request-id · mutation · audit signal| API Op -. bad tag .-> Rollback[scripts/staging_rollback.sh PREV_TAG] Rollback --> RP[rewrite CHARTNAV_IMAGE_TAG] RP --> UPapi[docker compose up -d api] UPapi --> Ready

Adapter resolution (phase 16)

```
flowchart LR Env[[env]] --> Cfg[app/config.py] Cfg -->|validates| Mode{CHARTNAV_PLATFORM_MODE} Mode -->|standalone| N[NativeChartNavAdapter] Mode -->|integrated_readthrough| Ad1{adapter key} Mode -->|integrated_writethrough| Ad2{adapter key} Ad1 -->|stub| S1[StubClinicalSystemAdapter writes_allowed=False] Ad1 -->|native| N Ad1 -->|vendor| VR[_VENDOR_ADAPTERS registry] Ad2 -->|stub| S2[StubClinicalSystemAdapter writes_allowed=True] Ad2 -->|native| N Ad2 -->|vendor| VR VR -->|key found| V[VendorAdapter] VR -->|key missing| ERR[RuntimeError: no registered adapter factory] N --> DB[(ChartNav DB)] V --> EXT[(External EHR/EMR)] S1 --> SM[(in-memory stub)] S2 --> SM
```

Core services consume the adapter through the `ClinicalSystemAdapter` protocol — they never reach the DB or a vendor SDK directly. `GET /platform` surfaces the resolved mode + adapter to the frontend so the admin UI can render a mode banner.

Release pipeline

```
flowchart LR TAG[["git push tag v*.*.*"]] --> RB[release.yml] RB --> V[resolve version] V --> DK[docker buildx push ghcr.io/.../chartnav-api:version + :latest] V --> RBS[scripts/release_build.sh] RBS --> API[chartnav-api-<v>.tar] RBS --> WEB[chartnav-web-<v>.tar.gz] RBS --> M[MANIFEST.txt] API --> GR[GitHub Release] WEB --> GR M --> GR DK --> OPS[operator: docker pull + compose up] GR --> OPS
```